08 · Multicore-Synchronisation & Safety

HSEM, Watchdogs und MPU-Grenzen · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie 1 / 53

Willkommen in der Welt der Robustheit!

- Achter Termin: Besitz, Sichtbarkeit, Frist und Speichergrenze
- Dieselbe Plattform: STM32H755, M7 Bare Metal, M4 FreeRTOS
- Vier Fähigkeiten: Lost Update, Warten ohne Verlust, Fensterrechnung, MPU-Grenze
- Lehrbeispiele sind keine Messung und keine Freigabe

Notizen 1 / 53

Vorlesung 7 hat gefragt, was eine Zeitmessung belegt. Zwei Programme können trotzdem dieselbe Stelle zerstören oder ihren Fortschritt verlieren, obwohl beide Takte laufen. Der Cortex-M7 arbeitet Bare Metal, also ohne Echtzeitkernel. Der Cortex-M4 nutzt FreeRTOS. CMSIS-RTOS2, Common Microcontroller Software Interface Standard in Version 2, ist die Schnittstelle zu diesem Kernel, nicht ein Scheduler über beide Kerne. CPU meint hier den jeweiligen Prozessorkern.
HSEM, Hardware Semaphore, ist ein Hardware-Schloss für eine Ressource. Ein Watchdog prüft, ob rechtzeitig aufgefrischt wird. Die MPU, Memory Protection Unit, prüft Regionen eines Kerns. Die vier Lernziele sind beobachtbar: ein Lost Update erklären, ein Warten ohne verlorenes Aufwecken skizzieren, ein Fenster aus dem zugehörigen Takt rechnen und eine MPU-Grenze benennen. Takte und die installierte HAL-Version, Hardware Abstraction Layer, sind offene Projektwerte. Sie hindern die Begriffe nicht.
Zuerst kommen Ressourcenkonflikte, danach Fehlererkennung und Schutz.

Folie 2 / 53

Agenda für heute

1. Gemeinsamer Zugriff und Lost Update
2. HSEM: Besitz, Warten, Recovery
3. Watchdog: Nachweis, IWDG, WWDG
4. EWI-Budget und MPU-Grenzen
5. Drei Laborteile, Beobachtung offen

Boot-HSEM und Nutzdaten-HSEM sind zwei IDs. ID 0 bleibt dem Start, wenn das Template sie nutzt.

Notizen 2 / 53

Die fünf Blöcke bauen aufeinander auf. Zuerst, wer eine gemeinsame Stelle anfassen darf. Dann, wie ein Schloss genommen, erwartet und im Fehlerfall nicht mit halben Daten weiterverwendet wird. Danach, welcher Nachweis einen Watchdog füttern darf. Anschließend, was eine Speichergrenze wirklich abfängt. Das Labor folgt derselben Trennung: Zähler, Fenster, kurze Reaktion und MPU-Test.
Aus Vorlesung 6 brauchen wir den nicht cachebaren Shared-Block, das Zustandsprotokoll und die Barriere. Aus Vorlesung 7 die Trennung von Beobachtung und Beweis. Wir wiederholen nicht die ganze Cache-Vorlesung. HSEM, Hardware Semaphore, ID 0 ist der Boot, wenn das Startprotokoll sie benutzt. ID 1 ist die Lehrressource für den Zähler, nur wenn sie frei ist. RTOS ist das Echtzeitbetriebssystem auf dem M4. Das Ping-Pong-Protokoll beantwortet bereits, wer wann schreiben darf.

Folie 3 / 53

Recap: Der Ping-Pong Datenaustausch

VL6, nicht „zwei Flags“ als Universalrezept. Ein Block `ipc` in SRAM4:

- `ipc.magic`, `ipc.state`, `ipc.payload`, Offsets 0, 4, 8
- Zustände `TURN_M7`, `REQUEST_M4`, `RESPONSE_M7`
- Payload schreiben, Barriere, dann Zustand veröffentlichen
- Basisregion nicht cachebar: `0x38000000`, 64 KiB

Notizen 3 / 53

Die vorige Folie hat zwei Schlösser getrennt. Hier steht der Datenvertrag, den das Schloss später umschließt. SRAM, Static Random Access Memory, ist der Speicher. SRAM4 ist die Bank ab `0x38000000`, 64 KiB, kein vierter Kern. `ipc` ist der gemeinsame Block aus Vorlesung 6, nicht ein zweites Layout. `volatile` hindert den Compiler, eine Pollschleife aus einem Register zu wiederholen. Es erzeugt keine Cache-Kohärenz und keine Sperre. DMB, Data Memory Barrier, ordnet sichtbare Speicherzugriffe. Es schreibt keine schmutzige Cache-Zeile zurück. Die Basisregion ist auf dem M7 nicht cachebar: TEX 001, C 0, B 0, S 1. Der M4 muss dieselbe Adresse erreichen. Die Attribute des M4 prüfen wir im Projekt.
Beim gemeinsamen Inkrement genügt das Ping-Pong nicht automatisch. Dort schreibt abwechselnd genau ein Berechtigter. `shared_count++` ist Lesen, Rechnen und Schreiben. Beide dürfen gleichzeitig lesen. Das Protokoll sagt für diesen Zähler nichts.
Als Nächstes zerlegen wir genau dieses Inkrement.

Folie 4 / 53

Das ungelöste Problem

- Ein FreeRTOS-Mutex gilt für Tasks dieses Kernels
- `__disable_irq` maskiert nur den lokalen Kern
- `shared_count++` ist Lesen, Addieren, Schreiben
- Software-Synchronisation zwischen Kernen ist möglich, hier aber nicht der Lehrweg
- HSEM ist die gewählte Hardware-TryLock-Lösung

Notizen 4 / 53

Das Ping-Pong hat einen Besitzer je Phase. Der Zähler hat zwei Schreiber ohne diese Phase. Ein Mutex ist ein gegenseitiger Ausschluss im lokalen Kernel. Der M7 ruft ihn nicht auf. `__disable_irq` sperrt Unterbrechungen des Kerns, der den Befehl ausführt. Der andere Kern läuft weiter. Read-Modify-Write heißt: alten Wert laden, im Register ändern, zurückschreiben. Zwischen den drei Schritten kann der andere Kern denselben Wert gelesen haben.
Ein Protokoll oder geeignete atomare Zugriffe können Kerne ebenfalls ordnen. Das ist keine generelle Unmöglichkeit von Software. Ob ein Zugriff auf diesem Chip atomar und sichtbar ist, hängt von Breite, Ausrichtung, Speicherattribut und Toolchain ab. `volatile` ist dafür kein portabler Ersatz. Wir wählen HSEM, Hardware Semaphore, weil der Erwerb in Hardware entschieden wird und der Vertrag im Labor kurz bleibt. HAL, Hardware Abstraction Layer, ist die Treiberschicht um die Register.
Welches Mittel welchen Konflikt abdeckt, zeigt die nächste Tabelle.

Folie 5 / 53

Warum ein FreeRTOS-Mutex hier versagt

Bedarf	Mittel
M7 gegen M4	HSEM plus Sichtbarkeit aus VL6
zwei M4-Tasks	lokaler Mutex, zusätzlich
ISR und Task, dieselbe Kennung	kein blockierender Mutex in der ISR

`HAL_HSEM_FastTake` setzt ProcessID 0. Zwei Nutzer mit derselben Kennung sind für HSEM ununterscheidbar.

Notizen 5 / 53

Die vorige Folie hat den lokalen Geltungsbereich genannt. Die Tabelle ordnet ihn zu. Ein FreeRTOS-Mutex bleibt richtig, wenn zwei Tasks des M4-Kernels denselben Puffer teilen. Er sieht den M7 nicht. Kommen beide Ebenen vor, gilt eine globale Sperrreihenfolge für lokale Mutexes und HSEM-IDs gemeinsam. ISR, Interrupt Service Routine, darf keine blockierende RTOS-API nehmen. RTOS ist das Echtzeitbetriebssystem, API die Programmierschnittstelle.

Notizen, Fortsetzung 5 / 53

FastTake ist das Ein-Schritt-Verfahren und trägt ProcessID 0. ProcessID ist eine softwaregewählte 8-Bit-Kennung, keine Core-ID und kein Task-Handle. Zwei M4-Tasks, die beide FastTake auf dieselbe ID rufen, kann das Schloss nicht trennen. Der einfache Versuch hat deshalb genau einen Worker je Kern. Eine ISR nimmt diese ID nicht nebenbei.
Frage: Warum schützt der M4-Mutex den M7 nicht? Weil der M7 das Protokoll dieses Kernels nicht benutzt.
Frage: Warum hilft `__disable_irq` auf dem M4 nicht gegen den M7? Weil der M7 weiterläuft.
Ohne diese Ebenen genügen zwei Inkremente für den Fehler.

Folie 6 / 53

Die Multicore Race Condition

Startwert 5. Zwei Inkremente. Erwartet 7. Die Schritte sind nacheinander, kein gemeinsamer Takt.

Reihenfolge	M7-Register	M4-Register	SRAM
M7 liest	5	—	5
M4 liest	5	5	5
beide rechnen	6	6	5
M7 schreibt	6	6	6
M4 schreibt	6	6	**6**

Geschützt und nacheinander: der zweite liest 6 und schreibt 7.

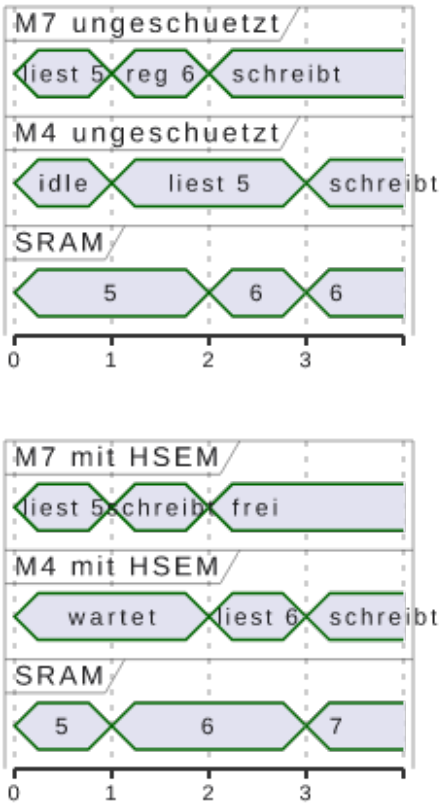
Notizen 6 / 53

Die Tabelle zeigt ein Lost Update, einen verlorenen Fortschritt. SRAM ist der gemeinsame Speicher, nicht das Register im Kern. Beide laden 5 in ihre private Kopie, addieren eins und schreiben 6. Zwei Schreibvorgänge ergeben 6, nicht 7. Die leeren Zellen der ersten Zeile sind „noch nicht gelesen“, nicht ein gleichzeitiger Takt. Die Reihenfolge ist didaktisch. Sie ist kein Trace und keine Nanosekundenmessung.
In der geschützten Variante hält der M7 das Schloss, liest 5, schreibt 6 und gibt frei. Der M4 nimmt danach neu, liest 6 und schreibt 7. Eine alte Registerkopie bleibt falsch, auch wenn das Schloss erst um das letzte Schreiben gelegt wird. Der Verlierer muss nach dem Erwerb neu laden.
Abbildung A stellt beide Verläufe nebeneinander.

Folie 7 / 53

Lost Update: zwei Inkremente, ein Ergebnis

Abbildung A. Schema, keine Messung. Links bleibt der SRAM bei 6. Rechts wird nach dem Erwerb neu gelesen.



Notizen 7 / 53

Abbildung A hat zwei Kernspuren und den gemeinsamen Speicher. Links lesen beide 5, beide bilden 6, der SRAM bleibt 6. Rechts wartet der M4, bis der M7 freigibt, und liest den neuen Wert. Die Kanten sind eine logische Reihenfolge. Die Achse ist kein Zeitmaßstab in Nanosekunden. HSEM, Hardware Semaphore, serialisiert hier den Erwerb. Die Sichtbarkeit des geschriebenen Wortes kommt zusätzlich aus der nicht cachebaren Region und der Barriere, nicht aus dem Schloss allein.
Frage: Warum reicht ein Schloss nur um das letzte Schreiben nicht? Weil die Registerkopie schon veraltet ist. Der Besitzer muss den Speicher nach dem Erwerb neu lesen und daraus rechnen.
Als Nächstes stehen die Eigenschaften, die dieses Schloss haben muss.

Folie 8 / 53

Die Anforderung: Ein Hardware-Schloss

- Exklusiver Erwerb, erkennbare Kennung, kurzer Abschnitt, geordnete Freigabe
- Der Hardwareentscheid ist atomar. Der HAL-Aufruf ist es nicht automatisch
- `HAL_OK`: Erwerb nach der Rückleseprüfung. `HAL_ERROR`: nicht bekommen
- Keine Warteschlange, keine Priority Inheritance, keine feste Latenz

Notizen 8 / 53

Die Abbildung hat den zweiten Schreiber warten lassen. Dafür brauchen wir vier Eigenschaften. Genau ein Besitzer hält die Ressource. Die Kennung ist prüfbar. Der geschützte Abschnitt bleibt kurz. Die Freigabe ist geordnet und gehört zum Besitzer. HSEM, Hardware Semaphore, liefert auf dieser Familie 32 Schlösser, IDs 0 bis 31. Das steht in RM0399, Kapitel Hardware Semaphore. Die installierte Revision des Handbuchs prüfen wir am Projekt. RM meint Reference Manual.
Atomar ist die Entscheidung im Schloss: nicht zwei Besitzer durch ein zerrissenes Lesen-Ändern-Schreiben des Lock-Worts. `HAL_HSEM_Take` schreibt und liest zurück. Das sind zwei Schritte der API, Application Programming Interface. `HAL_HSEM_FastTake` ist ein Schritt und ProcessID 0. Ein erfolgreicher HAL-Aufruf ist deshalb nicht „eine Busoperation“. TryLock heißt: sofort ja oder nein, ohne den Aufrufer in eine Faire-Warteschlange zu legen. Priority Inheritance gibt es hier nicht. Eine Erwerbszeit in Takten sagen wir nicht zu. `HAL_ERROR` ist ein fehlgeschlagener Erwerb, kein Busfehler und kein Deadlock. Das Peripheriegerät speichert den Besitz.

Folie 9 / 53

Einführung in Hardware Semaphores (HSEM)

Abbildung B. Frei oder belegt. Der Besitzer ist das Paar aus Core-ID und ProcessID. Recovery liegt daneben.



Begriff	Lehrwert	ist nicht
HSEM-ID	1, wenn frei	Task-Handle
ProcessID	`0x37` / `0x34`	Core-ID
Maske für ID 1	`1u << 1`	die Zahl 1

Notizen 9 / 53

HSEM heißt Hardware Semaphore. Der Automat hat zwei Zustände, die uns angehen: frei und belegt. Belegt trägt die Core-ID des Busmasters und die ProcessID, eine softwaregewählte Kennung von 0 bis 255. Reguläres Release verlangt dasselbe Paar. ReleaseAll ist ein anderer, schlüsselbasierter Weg für alle Schlösser, die der genannte Kern hält. Er steht nicht auf dem normalen Pfeil, weil er Nutzdaten nicht

Notizen, Fortsetzung 9 / 53

repariert. Ein HSEM-Schloss ist keine FreeRTOS-Counting-Semaphore und hat keine Task-Warteschlange. ID 0 reservieren wir, wenn der Boot aus Vorlesung 6 sie nutzt. ID 1 ist die Zählerressource nur, wenn sie frei ist. Die Lehrkennungen `0x37` für den M7 und `0x34` für den M4 sind ProcessIDs, keine Core-ID-Konstanten. Die Bitmaske der Notification für ID 1 ist `1u << 1`, also `0x00000002`, nicht die Maske 1.
Die nächste Folie trennt die drei Kennungen in den HAL-Funktionen.

Folie 10 / 53

HSEM Architektur im STM32H755

- `HAL_HSEM_Take(id, processId)`: schreiben, zurücklesen
- `HAL_HSEM_FastTake(id)`: ein Schritt, ProcessID 0
- Dieselbe Kennung noch einmal: Rückleseprüfung kann `HAL_OK` liefern
- Das ist kein Rekursionszähler und kein lokaler Taskbesitz

Aufruf	Ergebnis	Bedeutung
fremder Besitzer	`HAL_ERROR`	nicht bekommen
frei, passende ID	`HAL_OK`	neuer Besitz
selber Besitzer, noch einmal	oft `HAL_OK`	derselbe Besitz, nicht plus eins
ein Release	frei	kein „decrement“

Der öffentlich geprüfte STM32H7-HAL-Referenztreiber zeigt die Rückleseprüfung. Im Repo liegt dieser Treiber nicht. Die Cube-Version bleibt offen.

Notizen 10 / 53

Drei Namen bleiben getrennt. Die HSEM-ID ist die Ressource. Die Core-ID kommt aus der Hardwarezuordnung des Busmasters. Die ProcessID übergibt die Software an Take. `0x37` ist die Lehrkennung des M7-Workers, `0x34` die des M4-Workers. Frage: Warum ist `0x37` keine Core-ID? Weil wir sie wählen. Die Core-ID setzt der Zugriffsweg.
Take ist zwei Schritte: Register schreiben, Besitz zurücklesen. FastTake liest das Lock-Register in einem Schritt und verwendet ProcessID 0. Wenn derselbe Besitzer erneut Take ruft, kann die Rückleseprüfung denselben Zustand bestätigen und `HAL_OK` liefern. Es entsteht kein zweiter unabhängiger Besitz und kein Zähler, den ein Release herunterzählen würde. Ein normales Release gibt frei. Frage: Warum ist das kein rekursiver Mutex? Weil nichts mitzählt, wie oft genommen wurde.
Zwei lokale Tasks mit derselben Kennung unterscheidet HSEM nicht. Sie brauchen zusätzlich einen lokalen Mutex. Der einfache Versuch hat einen Worker je Kern.
Besitz stoppt den anderen Kern nicht und ordnet den Cache nicht.

Folie 11 / 53

Atomarität auf Silizium-Ebene

1. Take erfolgreich
2. Barriere, dann Nutzdaten
3. Barriere, dann Release
4. Sichtbarkeit bleibt der Vertrag aus VL6

DMB ordnet Zugriffe. Clean schreibt eine schmutzige Zeile zurück. Die Basisregion ist nicht cachebar. Beides nicht verwechseln.

Notizen 11 / 53

Der Hardwareentscheid und die Sichtbarkeit der Nutzdaten sind zwei Verträge. Nach erfolgreichem Take folgt eine Data Memory Barrier, DMB, damit der anschließende Zugriff auf `shared_count` nicht vor dem beobachteten Besitz sichtbar wird. Vor Release erneut DMB, damit das geschriebene Wort nicht hinter der Freigabe sichtbar wird. DSB, Data Synchronization Barrier, wartet auf den Abschluss. ISB, Instruction Synchronization Barrier, leert die Befehlspipeline. Für diesen Zähler brauchen wir die DMB an den zwei Stellen, nicht ein unkommentiertes DSB/ISB-Paar. Ob die HAL selbst Barrieren enthält, setzen wir nicht voraus. Der Referenztreiber ist nicht die installierte Cube-Version.
Die Region bleibt die aus Vorlesung 6: SRAM4, `0x38000000`, 64 KiB, auf dem M7 nicht cachebar. Ein DMB entfernt keine Dirty-Line. HSEM schaltet den Cache nicht kohärent. Eine cachebare Variante bräuchte einen eigenen Vertrag mit Clean, Invalidate, Ausrichtung und ohne False Sharing. Den zeigen wir hier nicht als Labor.
Gegenbeispiel: Der M7 hat 6 nur in einer Write-Back-Kopie. Der M4 liest im SRAM weiter 5, obwohl das Schloss inzwischen frei ist. Der nächste Besitzer inkrementiert die alte Sicht.
Der kleine Zählercode setzt genau diese Reihenfolge um.

Folie 12 / 53

Die HAL-API für HSEM

Lehrbeispiel, ungebaut. Ein Worker, ID 1 frei, Region nicht cachebar.

```
if (HAL_HSEM_Take(1, 0x37u) == HAL_OK) {
    __DMB();
    shared_count += 1u;
    __DMB();
    HAL_HSEM_Release(1, 0x37u);
} else {
    /* kein Besitz, Zaehler nicht anfassen */
}
```

`shared_count` ist ein ausgerichtetes `uint32_t` im Shared-Block, einmal vor beiden Workern auf 0 gesetzt. Im Abschnitt kein Delay und kein `printf`.

Notizen 12 / 53

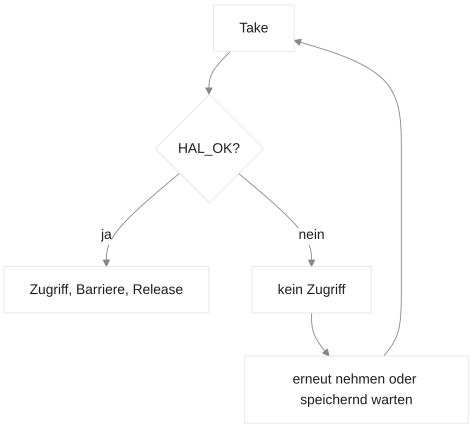
Jede Zeile hat eine Rolle. Die ID 1 ist die Ressource, `0x37` die ProcessID des M7, nicht die Core-ID. `HAL_OK` heißt, die Rückleseprüfung hat den Besitz gesehen. Danach ordnet `__DMB` den folgenden Schreibzugriff hinter diesen Erwerb. `shared_count += 1u` ist nur innerhalb dieses Besitzes das geschützte Inkrement. Die zweite Barriere liegt vor Release, damit die Freigabe den neuen Wert nicht überholt. Release verwendet dieselbe ID und dieselbe ProcessID. Im `else` gibt es keinen Nutzdatenzugriff. `HAL_ERROR` ist der nicht erhaltene Besitz.

Der Zähler liegt im gemeinsamen, nicht cachebaren Block. Die genaue Adresse ist die aus Vorlesung 6 beziehungsweise ein offener Projektparameter, keine ausgedachte Linker-Adresse. Der Abschnitt bleibt das eine Inkrement. Eine Ausgabe oder ein Warten darunter würde das Schloss festhalten und andere aushungern. HSEM ist nicht fair. Busy-Wait kann den Bus belasten. Der M4 verwendet `0x34` an derselben ID.

Erwerb kann scheitern. Daraus wird die Frage, wie man wartet, ohne eine Freigabe zu verlieren.

Folie 13 / 53

Flowchart: HSEM Take



Ein dritter Teilnehmer kann nach einer Freigabe zuerst gewinnen. `HAL_ERROR` ist kein Fault.

Notizen 13 / 53

Das Diagramm ist die Entscheidung, kein Zeitverlauf. Erfolg bedeutet: Daten anfassen, Barriere, freigeben. Fehlschlag bedeutet: die Nutzdaten nicht anfassen. Danach darf der Aufrufer erneut versuchen oder auf ein speicherndes Ereignis warten. Vor jedem Zugriff wird der Besitz neu geprüft. Eine Notification, also die Meldung einer Freigabe, ist kein Besitz. Ein dritter Kern oder Worker kann zwischen Freigabe und eigenem Take gewinnen.
`HAL_ERROR` aus Take ist der nicht erhaltene Erwerb. Es ist kein MemManage, kein HardFault und kein Beweis für einen defekten Bus. Ein HardFault wäre eine Exception, keine Rückgabe dieser Funktion. Polling, wiederholtes Fragen, belastet Kern und Bus. Ein Interruptweg ist nicht automatisch überlegen. Er hat eigene Lücken, wenn das Aufwecken verloren geht.
Diese Lücke ist der nächste Ablauf.

Folie 14 / 53

Warten auf den HSEM: Polling vs. Interrupts

Falsch: Take scheitert, der andere gibt frei, erst dann wird die Notification scharf, danach wird unendlich gewartet. Die Freigabe ist weg.
Richtig, ein wartender M4-Task, speicherndes Thread-Flag, eine Gesamtdeadline:

1. Handle gültig, alte Ereignisse der eigenen ID klären
2. Notification der Maske `1u << 1` aktivieren
3. Take erneut
4. Erfolg: Notification aus, kurz arbeiten
5. Fehler: Flag nicht löschen, mit Restzeit warten
6. Nach dem Wecken erneut scharf und erneut Take
7. Timeout: aufräumen, nicht anfassen

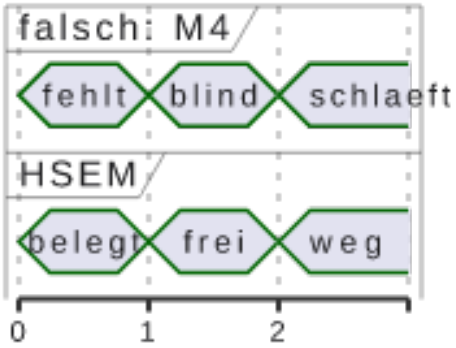
Notizen 14 / 53

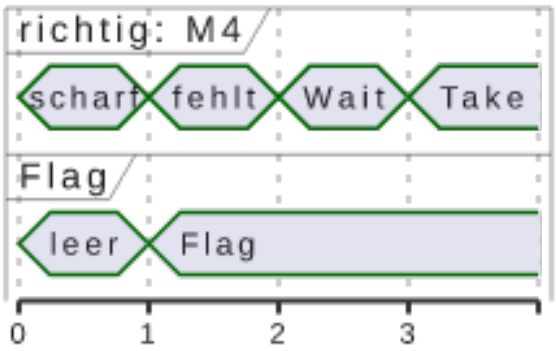
Polling fragt in einer Schleife. Es erkennt den freien Zustand, solange es läuft, kostet aber CPU und Bus. Der Interruptweg soll schlafen. IRQ ist die Unterbrechungsanforderung. Das falsche Muster verliert das Aufwecken, Lost Wakeup: Die Freigabe passiert, bevor jemand sie speichert. Abbildung C zeigt denselben M7-Besitzer, dasselbe Schloss und denselben M4-Wartenden in beiden Spuren.
Das Flag ist ein RTOS-Thread-Flag des wartenden Tasks. Es wird nicht zwischen dem letzten fehlgeschlagenen Take und dem Wait gelöscht. Sonst kann ein Callback in genau dieser Lücke das neue Signal wieder austragen. Aufwecken beweist keinen Besitz. Es erlaubt höchstens einen neuen Take. Ein veraltetes Flag auch. Die Deadline ist eine Frist für den ganzen Versuch. Jeder neue IRQ setzt sie nicht auf den vollen Wert zurück. Die Restzeit bleibt `deadline - jetzt`.
Der Referenz-HAL-Handler schaltet ausgelöste Notification-Bits ab, löscht die Flags und ruft den Callback mit einer Semaphormaske. Deshalb muss der Wartepfad danach neu scharfschalten. Die Cube-Version im Projekt kann abweichen. Wir zeichnen das Rearming deshalb ausdrücklich.
Der konkrete Pfad vom M7-Release bis zum Flag folgt jetzt.

Folie 15 / 53

HSEM Interrupts: M7 weckt M4-Task

Abbildung C. Oben geht die Freigabe verloren. Unten liegt sie im Flag, bevor gewartet wird.





Lehrhinweis, ungebaut: Callback setzt das Flag des vorher gültigen Handles. Keine RTOS-Blockierung in der ISR. NVIC-Priorität auf der erlaubten Seite der Syscall-Grenze. Niedrigere Zahl ist höhere Dringlichkeit.

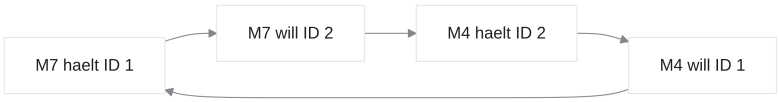
Notizen 15 / 53

Der M7 gibt mit passender Kennung frei. Die Hardware setzt den Status der ID. Der NVIC, Nested Vectored Interrupt Controller, des M4 nimmt den HSEM-IRQ an, wenn er freigegeben und nicht maskiert ist. Der Referenzhandler deaktiviert die ausgelösten Notification-Bits, löscht deren Flags und übergibt dem Callback die Maske. Für ID 1 ist das Bit 1, Maske `0x00000002`. Der Callback ruft eine ISR-taugliche Funktion, zum Beispiel `osThreadFlagsSet` auf das Handle des einen wartenden Tasks. Ein nackter Task-Name ist kein Handle. Das Handle existiert, bevor die Notification aktiv wird.
In der ISR wird nicht gewartet und kein Mutex genommen. Ob der Aufruf erlaubt ist, hängt von der Priorität relativ zur Syscall-Grenze des Ports ab. Die Grenze steht im verwendeten FreeRTOS-Port, nicht als erfundene Zahl. Nach dem Aufwecken schaltet der Task die Notification neu scharf und ruft Take. Verliert er, wartet er mit der restlichen Deadline weiter.
Frage: Was garantiert die Notification? Dass eine Freigabe beobachtet wurde. Den neuen Besitz holt nur Take.
Ein Aufwecken verhindert noch keinen zyklischen Lockkonflikt.

Folie 16 / 53

Multicore Deadlock-Gefahr

Zwei Ressourcen, zwei Kerne, entgegengesetzte Reihenfolge.



Globale Ordnung, zum Beispiel immer ID 1 vor ID 2, und lokale Mutexes in derselben Ordnung. Timeout ohne Freigabe der eigenen Schlösser beendet den Zyklus nicht.

Notizen 16 / 53

Deadlock heißt: jeder wartet auf etwas, das ein anderer in der Runde hält, und niemand kommt weiter. Die vier Coffman-Bedingungen sind hier sichtbar. Wechselseitiger Ausschluss: jede ID hat einen Besitzer. Halten und Warten: der M7 hält ID 1 und will ID 2. Kein Entzug: ein normaler Take nimmt dem Besitzer nichts weg. Zyklisches Warten: die Kanten schließen sich. Fehlt eine Bedingung, entsteht dieser Zyklus so nicht. Die Gegenmaßnahme im Entwurf ist eine globale Reihenfolge für alle Schlösser, einschließlich der lokalen FreeRTOS-Mutexes. Wer ID 2 braucht, hält ID 1 schon oder nimmt sie zuerst. Nie umgekehrt. Der andere Kern und unbeteiligte Tasks laufen weiter. Das ist kein Stillstand des Chips. Ein Timeout am Warten beendet nur das Warten. Das bereits gehaltene Schloss bleibt, bis es freigegeben oder der Versuch mit Rollback abgebrochen wird. Rollback heißt: eigene Teilschritte zurücknehmen und eigene Schlösser freigeben. Halb geschriebene Nutzdaten bleiben sonst liegen.
Wie man auf einen vermissten Besitzer reagiert, ist die nächste Grenze.

Folie 17 / 53

Recovery: Schlossfreigabe repariert keine Daten

`HAL_HSEM_ReleaseAll` gibt mit einem Key alle vom genannten Kern gehaltenen HSEM frei. Das ist kein normales Release einer ID und kein kryptografischer Nachweis.
Abbildung D. Verdacht ist kein Beweis, dass CPU, ISR und DMA aufgehört haben zu schreiben.



Kein „Timeout, ReleaseAll, weiterzählen“.

Notizen 17 / 53

Der Titel der älteren Folie klang nach einer Lösung. ReleaseAll repariert keine Daten. Ein Key ist die technische Freigabebedingung im Register, kein Sicherheitsbeweis. Ein fehlender Heartbeat, ein ausbleibendes Lebenszeichen, ist ein Verdacht. Der andere Kern kann in einer ISR stehen, und DMA, Direct Memory Access, kann als eigener Busmaster weiter schreiben. Geben wir die Schlösser frei, während das noch passiert, können zwei Seiten sich für Besitzer halten. Teilweise geschriebene Felder werden dann veröffentlicht.
Wenn wir den Besitzer nicht zuverlässig anhalten und den Speicher nicht prüfen können, ist der Lehrpfad ein geordneter System-Reset, eine neue Initialisierung des Shared-Blocks und der Handshake von vorn. System-Reset ist nicht der Verlust jedes Backup- oder Debug-Zustands. Die Anwendungsdaten im Shared-Block gelten danach als ungültig, bis der Boot sie neu aufsetzt. Ein sicherer Ausgangszustand der Anlage hängt von der Anwendung ab. Den erfinden wir hier nicht.
Frage: Warum ist ReleaseAll nach einem Timeout keine allgemeine Reparatur? Weil der vermeintliche Besitzer oder DMA weiter schreiben kann und halbe Daten inkonsistent bleiben.
Die Zusammenfassung setzt die Voraussetzungen in eine Reihenfolge.

Folie 18 / 53

Zusammenfassung Hardware-Semaphoren

- Lokal serialisieren, dann Inter-Core-Besitz nehmen
- Sichtbarkeit aus VL6, kurz arbeiten, passend freigeben
- Warten nur mit gespeichertem Ereignis und erneutem Take
- ReleaseAll ist Recovery, nicht der Ping-Pong

Frage: Warum ist eine Notification kein Besitz? Weil danach Take verlieren kann. Frage: Warum ersetzt die ProcessID keinen Task-Mutex? Weil dieselbe Kennung zwei Tasks nicht trennt.

Notizen 18 / 53

Der gedankliche Ablauf ist eine Kette. Zwei Tasks eines Kerns serialisiert der lokale Mutex. Den anderen Kern schließt das HSEM aus, wenn die Kennungen verschieden sind und nur ein Worker je Kern nimmt. Nach dem Erwerb gilt die nicht cachebare Region und die DMB um den Zähler. Der Abschnitt bleibt kurz. Release benutzt dieselbe ProcessID. Wer warten muss, schaltet die Notification scharf, versucht Take

Notizen, Fortsetzung 18 / 53

erneut, löscht das Flag nicht vor dem Wait und behandelt Aufwecken als Anlass für einen neuen Take. Die Gesamtdeadline wird nicht bei jedem IRQ verlängert.
Die zwei Fragen haben feste Antworten. Die Notification meldet eine Freigabe, nicht den neuen Besitzer. Die ProcessID ist eine 8-Bit-Kennung des Aufrufs, kein FreeRTOS-Handle. Dieselbe Kombination aus Core-ID und ProcessID ist für das Schloss ein Nutzer. Recovery bleibt die Grenze der vorigen Folie: ohne gesicherte Ruhe des alten Besitzers kein Weiterbetrieb auf halben Daten.
Wenn der Fortschritt ausbleibt, braucht es zusätzlich eine Fehlererkennung.

Folie 19 / 53

Was ist „Safety“ in Embedded Systems?

Safety ist der Schutz vor unvertretbaren Folgen einer Fehlfunktion. Security, der Schutz vor absichtlichem Missbrauch, ist ein anderes Ziel und hier nicht das Thema.
Ein Lehrbeispiel, keine Anlage: ein Ausgang muss innerhalb einer genannten Frist einen definierten Pegel annehmen. Ob das 10 ms oder 1 s sind, legt die Anwendung fest. Dieselben Zahlen sind hier nur Platzhalter.
`100` ist `0x0064`. Bit 15 gesetzt ergibt `0x8064` = 32868, nicht 65535 und nicht die Umkehrung aller Bits.

Notizen 19 / 53

Die Schlösser haben Konkurrenz geordnet. Sie haben nicht geprüft, ob die Arbeit noch lebt oder das Ergebnis stimmt. Safety bezeichnet hier diese zweite Frage. Ein Lock, ein Watchdog oder eine MPU sind Mechanismen. Ein Sicherheitsnachweis wäre eine argumentierte Kette aus Anforderung, Maßnahme und Prüfung. Die haben wir für keine konkrete Maschine. „Normen verlangen immer einen Reset“ sagen wir nicht.
Das Bitbeispiel zeigt, warum eine Plausibilitätsprüfung nötig sein kann. Ein `uint16_t` mit dem Wert 100, hexadezimal `0x0064`, bleibt 100, solange nur die unteren Bits gemeint sind. Setzt ein Fehler Bit 15, entsteht `0x8064`, dezimal 32868. Das ist kein Überlauf auf 65535. Die Prüfung kann in Software den Wertebereich vergleichen. Sie kann auch eine unabhängige Zeitgrenze nutzen. Beides erkennt Verschiedenes.
Der nächste Block ordnet Fehler und die Frage, ob ein Watchdog sie sieht.

Folie 20 / 53

Ursachen für System-Crashes

Fehler	Watchdog sieht ihn?
CPU-Hog, niemand füttert	ja, die Frist läuft ab
Worker tot, Idle füttert	oft nein
Ergebnis falsch, Takt stimmt	nein
anderer Kern tot, dieser füttert	nein

Drei Sätze bleiben getrennt: der Handler läuft, die Runde ist abgeschlossen, das Ergebnis ist richtig.

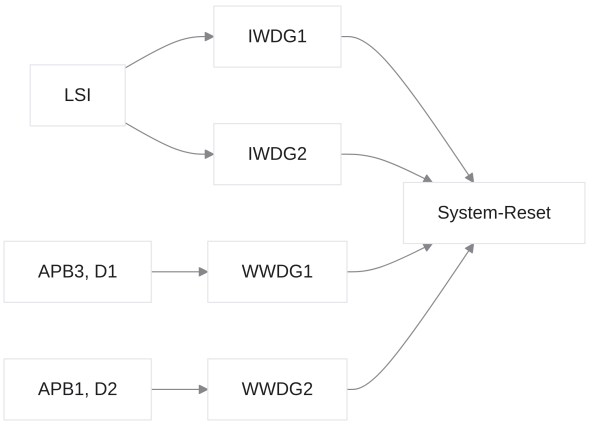
Notizen 20 / 53

Ein CPU-Hog belegt den Kern, ohne den Watchdog aufzufrischen. Wenn kein anderer Pfad füttert, läuft die Frist ab. Ein Deadlock zweier Tasks lässt die Idle-Task weiterlaufen. Füttert Idle, bleibt der Fehler unsichtbar. Eine Schleife, die pünktlich füttert und fachlich falsch rechnet, besteht die Zeitprüfung. Stirbt der andere Kern, während dieser Kern weiter einen frischen eigenen Nachweis meldet und füttert, sieht der Hund den fremden Ausfall nicht. Die Spalte hängt also an der Refresh-Policy, nicht am Baustein allein. Heartbeat ist ein Lebenszeichen. Fortschritt ist eine abgeschlossene Runde. Ein richtiges Ergebnis ist eine fachliche Prüfung. Der Watchdog prüft nur, ob rechtzeitig aufgefrischt wurde. Die Software muss ihm die Gesundheitsbedingung liefern. Abbildung F zeigt dieselben drei Verläufe gleich anschließend beim Fütterungsort.
Der Hardwarezähler liefert die Frist. Die Software entscheidet, wann sie ihn zurücksetzt.

Folie 21 / 53

Das Konzept des Hardware-Watchdogs

Der Zähler läuft abwärts. Refresh setzt ihn in den erlaubten Bereich zurück. Ohne rechtzeitigen Refresh folgt ein Reset.
Auf dem STM32H755 sind IWDG1, IWDG2, WWDG1 und WWDG2 dokumentierte Quellen eines System-Resets. Das steht in RM0399, Rev. 5, bei den RCC-Resetursachen. RCC ist Reset and Clock Control.
System-Reset löscht nicht jeden Backup- und Debug-Zustand. Die Resetflags sichern, bevor Boot oder HAL sie löschen.
Abbildung E, Taktpfade. Der Resetpfeil ist der System-Reset, nicht eine offene Kernfrage.



Notizen 21 / 53

Ein Hardware-Watchdog ist ein eigener Zähler. Die Software muss ihn innerhalb der konfigurierten Frist bedienen. Sonst löst er aus. IWDG, Independent Watchdog, hängt am internen langsamen Takt. WWDG, Window Watchdog, hängt am APB seiner Instanz. APB ist der Advanced Peripheral Bus. D1 und D2 sind Strom- und Takt-Domänen des STM32H7, keine eigenen Resetklassen. Die Zuordnung erklärt, welcher Takt gilt. Sie ändert nicht die dokumentierte Wirkung: alle vier Instanzen können einen System-Reset auslösen. EWI, Early Wakeup Interrupt, ist eine Unterbrechung vor dem späten Ablauf und kein anderer Resetumfang.
System-Reset startet die Kerne neu und setzt viel Peripherie zurück. Backup-Register und Debug-Logik können erhalten bleiben. Pins können hochohmig oder analog werden. Ob ein Ausgang dabei sicher ist, hängt von externen Pull-Widerständen und der Bootsequenz ab. Der Reset beweist weder die Fehlerursache noch einen sicheren Anlagenzustand. Die Flags in RCC unterscheiden die Quelle nur, wenn wir sie lesen, bevor sie gelöscht werden.
Was die Software zum Refresh berechtigt, ist die nächste Pflicht.

Folie 22 / 53

Die Pflicht der Software (Kicking)

Kick oder Refresh heißt: den Watchdogzähler in den erlaubten Bereich zurücksetzen. Im Entwurf nur nach einem abgeschlossenen, noch frischen Nachweis.
Nicht „niemals in der ISR“. Eine ISR, die immer füttert, verdeckt einen Hänger. Eine ISR darf füttern, wenn sie die Gesundheitsbedingung wirklich geprüft hat und die API dort erlaubt ist.
Unterschiedliche Workerperioden brauchen eigene Fristen. Ein gemeinsamer Haken „beide neu seit der letzten Abfrage“ ist ein engeres Modell.

Notizen 22 / 53

Die Hardware zählt, ob gefüttert wurde. Sie weiß nicht, ob der Regler einen Schritt beendet hat. Der Supervisor, eine prüfende Stelle, liefert diese Bedingung. Ein Lebenszähler des Workers steigt nach einer abgeschlossenen Runde, einschließlich der vorgesehenen Prüfungen. Eine Runde ohne neues Sensorereignis kann trotzdem gültiger Fortschritt sein. Ein Hochzählen am Schleifenanfang oder in einer ISR ohne abgeschlossene Arbeit ist keiner.
Ein Bitfeld, das man liest und später ganz löscht, verschluckt Bits, die dazwischen gesetzt wurden. Ein atomares Exchange, ein Tausch in einem Zugriff, vermeidet das nur, wenn der Zugriff für diesen Speicher und diese Kerne wirklich atomar ist. Im Lehrbeispiel schreiben zwei Kerne getrennte Wörter. Jeder ist der einzige Schreiber seines Zählers. Zwei getrennte Lesevorgänge sind kein Schnappschuss eines Paares. Die Sichtbarkeit bleibt der Vertrag aus Vorlesung 6.
Frage: Warum ist eine unveränderte Epoche bei einer schnellen Abfrage kein automatischer Fehler? Weil die erlaubte Workerperiode länger sein darf als der Abstand der Abfragen. Entscheidend ist die Frischefrist.
Der erste Baustein rechnet diese Frist aus dem LSI.

Folie 23 / 53

Watchdog Typ 1: Independent Watchdog (IWDG)

$$t_{\text{nom}} = (R+1) \times P / f_{\text{LSI}}$$

Lehrannahme: $R = 500$, $P = 32$, $f_{\text{LSI,nom}} = 32000\,\text{Hz}$.

$$501 \times 32/32000 = 0{,}501\,\text{s} = 501\,\text{ms}$$

Frühestes Ende bei der höchsten LSI-Frequenz, spätestes bei der niedrigsten:

$$t_{\text{min}} = (R+1)\,P/f_{\text{LSI,max}}, \quad t_{\text{max}} = (R+1)\,P/f_{\text{LSI,min}}$$

Die Zahlengrenzen stehen im Datenblatt zur Temperatur und Versorgung. DS12919 Rev. 3 trennt unter anderem eine 25-°C-Zeile von breiteren Bereichen. Ein Raumtemperaturwert ist kein Worst Case. Im Hauptbeispiel bleibt das Fenster des IWDG aus. Die Familie kann ein Fenster. Das ist ein getrenntes Beispiel.

Notizen 23 / 53

LSI, Low-Speed Internal, ist ein interner RC-Oszillator, kein Quarz. RC meint Widerstands-Kondensator-Schaltung. Die Frequenz streut mit Temperatur und Versorgung. R ist der Reload, der Nachladewert. P ist der Vorteiler. Die Formel teilt die Anzahl der LSI-Takte durch die Frequenz und ergibt Sekunden. 501 mal 32 ist 16032. Geteilt durch 32000 ist 0,501 s, also 501 ms. Das ist der Nennwert, keine Messung und keine garantierte Deadline. Dazu kommen Übernahme der Register, Quantisierung und eine Refresh-Marge, ein Abstand zu beiden Enden.
Warum die höchste Frequenz früher abläuft: derselbe Zählerstand wird in kürzerer Zeit erreicht. Die konkreten Minima und Maxima übernehmen wir nur mit der Zeile des Datenblatts, die zur Bedingung passt. Sie stehen nicht als eine einzige Zahl in dieser Folie, solange die Projektbedingungen fehlen.
Der STM32H755-IWDG hat eine Fensteroption. Für das einfache Beispiel schalten wir sie aus und sagen das dazu. Sonst würde ein zu früher Refresh je nach Betrieb ebenfalls auslösen. „IWDG grundsätzlich ohne Fenster“ wäre falsch.
Die Takttrennung ist nützlich und begrenzt.

Folie 24 / 53

Der Vorteil des LSI

Der IWDG hängt nicht am HSE, High-Speed External, dem externen schnellen Takt. Ein Ausfall dieses Takts ist nicht automatisch ein Ausfall der Versorgung oder des LSI.
Clock-Security und Fallback können den CPU-Takt auf eine andere Quelle legen. Ob das konfiguriert ist, steht im Projekt. Gemeinsame Fehler, zum Beispiel die Versorgung, treffen LSI und Kern zusammen. „Unabhängig“ heißt hier: anderer Taktpfad, nicht universelle Rettung.

Notizen 24 / 53

Abbildung E hat den LSI neben den APB-Takten gezeichnet. Fällt der externe Takt aus, kann der IWDG weiterzählen, solange sein Oszillator und die Versorgung leben. Der Kern kann gleichzeitig stehen oder auf einem Fallback weiterlaufen. Beides ist nicht dasselbe Bild. Eine Clock-Security-Funktion meldet den Ausfall des überwachten Takts und kann umschalten. Wir behaupten sie nur, wenn das Projekt sie einschaltet. Die genaue Wirkung steht im RCC-Kapitel der gelesenen Revision, nicht als pauschale Rettung.
Ein Watchdog, dessen Takt mit ausgefallen ist, rettet diesen Ausfall nicht. Darum ist der getrennte Pfad der Vorteil und zugleich die Grenze. Für das Labor bleibt der IWDG das lange Timeout mit ausgeschaltetem Fenster. Der WWDG bleibt das kurze Fenster am APB.
Wirksam wird das nur, wenn Start und Registerübernahme stimmen.

Folie 25 / 53

Konfiguration des IWDG

Reihenfolge als Lehrfolge, Symbole am Header der vorhandenen HAL prüfen:

1. Instanz wählen, LSI verfügbar
2. Prescaler und Reload schreiben
3. Fenster für dieses Beispiel aus
4. Start, auf die Übernahmebits warten, dann Refresh
5. Debug-Freeze bewusst an oder aus
6. RCC-Resetursache lesen, bevor sie gelöscht wird

Rückgabe prüfen. Ein stiller Fehler beim Start ist kein laufender Hund.

Notizen 25 / 53

Die beste Formel hilft nicht, wenn der Zähler nie startet oder der Debugger ihn einfriert. Die Instanz ist IWDG1 oder IWDG2. Welche zum Versuch gehört, tragen wir ein. Der Prescaler und der Reload sind die Werte der 501-ms-Rechnung nur, wenn wir genau dieses Beispiel fahren. Das Fensterregister lassen wir im Hauptbeispiel unbenutzt und dokumentieren das. Der Referenztreiber aktualisiert manche Register nur nach einer Statusprüfung. Die Reihenfolge Start und Refresh nicht umdrehen, ohne den Treiber gelesen zu haben. Die installierte HAL liegt nicht im Repo. Option-Bytes, die den Watchdog schon aus dem Reset

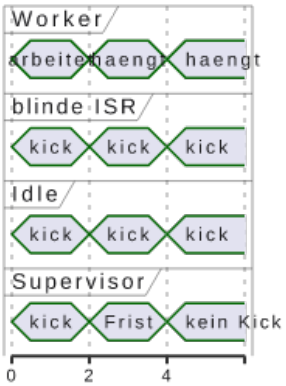
Notizen, Fortsetzung 25 / 53

starten, behaupten wir nicht.
Debug-Freeze hält den Zähler an, solange der Kern steht. Eine Messung mit Debugger ist dann ein anderer Versuch als ohne. Low-Power kann den Takt ebenfalls ändern. Das gehört in die Bedingung, nicht in eine vermutete Option. Die Resetflags sichern wir in einer frühen Stelle des Starts. Eine spätere HAL-Init kann die Historie löschen. SRAM-Inhalt und der M7-Cache überleben einen Reset nicht automatisch als gültiger Logsatz.
Die Konfiguration ist nutzlos, wenn die falsche Stelle füttert.

Folie 26 / 53

Die Gefahr im RTOS: Wo füttern wir?

Dieselbe hängende Worker-Runde, drei Policies. Abbildung F.



Ein Watchdog-Task, der nur existiert, genügt nicht. Er muss die Nachweise sehen und selbst rechtzeitig laufen.

Notizen 26 / 53

Die drei Streifen starten mit einem gesunden Worker und demselben Ausfall. Die blinde Timer-ISR füttert weiter, weil ihr Handler lebt. Idle füttert weiter, weil dieser Pfad CPU-Zeit bekommt, obwohl die Worker blockiert sind. Der Supervisor stellt fest, dass der Nachweis älter ist als die Frist, verriegelt den Fehler und füttert nicht mehr. Die Zeiten auf der Achse sind Schritte des Schemas, keine Produkt-Deadlines.
Der Supervisor ist selbst eine Task oder ein geprüfter Kontext. Wenn er nicht läuft, füttert niemand aus dem richtigen Grund, und der Hund läuft ab. Das ist erwünscht, sobald die Frist seines eigenen Ausfalls eingehalten ist. Sieht er den anderen Kern nicht, weil die Sichtbarkeit fehlt, darf er nicht so tun, als sei der fremde Zähler frisch. Unterschiedliche Perioden und ein Startup-Budget, bevor der erste vollständige Nachweis da ist, gehören zum nächsten Code.
Zuerst das ISR-Muster im Kleinen.

Folie 27 / 53

Anti-Pattern: Füttern im Timer-Interrupt

```
void TIM_IRQ(void) {
    kick(); /* auch wenn der Worker steht */
}
```

Der Handler beweist, dass dieser IRQ bedient wird. Er beweist nicht, dass eine Anwendungsschleife eine Runde beendet hat.

Notizen 27 / 53

Der Code ist absichtlich kurz. `TIM_IRQ` steht für eine periodische Interruptroutine. `kick` steht für den Refresh des Watchdogs. Beides sind Lehrnamen, keine geprüften Symbole des Projekts. Die Routine läuft, solange der Takt der Timerquelle läuft, die Priorität das zulässt und der Handler nicht selbst blockiert. Ein Regler, der in seiner Task auf ein Schloss wartet, ändert daran nichts. Die Kicks gehen weiter. Der Hund sieht einen gesunden Takt.
Daraus folgt nicht, dass jeder Refresh in einer ISR falsch ist. Falsch ist der Refresh ohne die vereinbarte Gesundheitsbedingung. Wenn der Handler die Nachweise selbst geprüft hat, die Prüfung kurz ist und die Priorität stimmt, kann der Ort die ISR sein. Im Labor ist das nicht das Beispiel. Wir zeigen den verdeckten Hänger, damit die Policy sichtbar wird.
Dieselbe Grenze hat das Füttern in der Idle-Task, mit einem anderen falschen Beweis.

Folie 28 / 53

Begrenzt: Füttern in der Idle-Task

Idle-Ausführung zeigt: dieser lokale Pfad bekommt CPU-Zeit. Sie zeigt keinen neuen Scheduler-Tick und keinen Workerfortschritt.
Zwei Tasks können aufeinander warten. Idle läuft weiter und würde weiter füttern. Tickless Idle kann sogar ohne periodischen Tick in diesen Pfad kommen. Ein Idle-Hook darf nicht blockierend aufrufen.

Notizen 28 / 53

Die vorige ISR hat den Handler mit der Anwendung verwechselt. Idle verwechselt „der Scheduler konnte etwas Niedriges ausführen“ mit „die Arbeit ist vorangekommen“. Der frühere Satz, Idle beweise einen Tick, stimmt nicht. Der Kernel kann Idle auch im ticklosen Leerlauf betreten. Ein Tick ist der periodische Zeittakt des RTOS, nicht die Idle-Schleife.
Konkretes Gegenbeispiel: Task Hoch hält ein Schloss und wird von einer mittleren Aufgabe verdrängt, Task Niedrig wartet auf das Schloss. Beide Worker kommen nicht voran. Idle läuft, sobald die mittlere Aufgabe wartet oder endet, und ein Hook darin füttert. Der Watchdog sieht kein Problem. Das ist eine schwache Policy, kein gelungener Nachweis. Idle beweist nur den eigenen Ausführungspfad.
Wenn ein Hook gezeigt wird, gelten die Grenzen des Ports: kein `osDelay`, kein Warten auf ein Schloss,

Notizen, Fortsetzung 28 / 53

kurze Arbeit. Für blockierte Worker brauchen wir deren eigene Zähler.
Diese Zähler liest der Supervisor.

Folie 29 / 53

Supervisor mit Fortschrittsfristen

Gegenbeispiel, Kohorte, Start `seen = 0`. Unbedingt nach der Prüfung zu speichern, verbraucht Fortschritt:

- Abfrage A: `e4 = 1, e7 = 0`. Gemeinsam nicht frisch.
- Setzt A trotzdem `seen4 = 1`, wirkt Abfrage B mit `e4 = 1, e7 = 1` wie „M4 ohne neuen Schritt“.

Richtig im Kohortenmodell: `seen` erst nach akzeptiertem gemeinsamen Nachweis nachziehen.
Das ist die Vereinfachung „beide neu seit dem letzten Erfolg“.
Hauptmodell bei verschiedenen Perioden: jeder Worker hat einen eigenen Zähler und eine eigene Frischefrist. Ein einmal unveränderter Zähler in einer schnellen Abfrage ist noch kein Fehler.

Notizen 29 / 53

Die alte Folie hat `seen_m4` und `seen_m7` auch nach einem fehlgeschlagenen gemeinsamen Test überschrieben. Damit gilt der Fortschritt des schnelleren Workers als verbraucht, obwohl noch nicht gefüttert wurde. Die nächste Abfrage verlangt von ihm einen weiteren Schritt, den die Policy nicht verlangen dürfte. Die Korrektur der Kohorte speichert die gesehenen Werte nur im erfolgreichen Zweig. Eine erfolglose Abfrage lässt `seen` stehen. Die Bedingung bezieht sich auf die letzte akzeptierte Kohorte. Für ungleiche Perioden ist das Kohortenmodell zu grob. Ein schneller Worker darf mehrere Runden machen, während der langsame noch in seiner Frist liegt. Der Supervisor liest je Worker ein Wort, als einziger Beobachter seiner Kopie. Ändert sich der Wert, merkt er den neuen Stand und die lokale Zeit `now`. Die Zeitbasis ist die des Supervisors, nicht eine mit dem anderen Kern synchronisierte Uhr. Ein Zeitstempel im Shared-Block würde diese Synchronisation extra brauchen. Wird `now - seen_at` größer als die Frist, wird der Fehler verriegelt und nicht mehr normal gefüttert. Vor dem ersten Nachweis gilt ein Startup-Budget. Danach ist fehlende Meldung ein Fehler.
Dieselbe frische Runde darf mehrere Abfragen überstehen, solange die Frist reicht. Es ist nicht behauptet, dass jeder Refresh eine neue gemeinsame Epoche verbraucht. Die Policy muss zur maximalen Reaktionszeit passen: Frist plus restlicher Watchdog plus Reaktion.
Der Code steht auf der nächsten Unterfolie, damit die Zeilen sichtbar bleiben.

Folie 30 / 53

Fristenbeispiel, zu Folie Supervisor

Lehrpseudocode, ungebaut. Ein Schreiber je Zähler. `now` ist die lokale Zeit des Supervisors.

```
if (cur != w->last) {
    w->last = cur;
    w->seen_at = now;
    w->armed = 1;
}
if (!w->armed)
    return now <= startup_deadline;
return (uint32_t)(now - w->seen_at) <= w->fresh;
```

Refresh nur, wenn jeder nötige Worker so frisch ist und das Hardwarefenster offen ist. Sonst Fehler verriegeln und nicht füttern.

Notizen 30 / 53

`cur` ist die stabile Kopie eines `uint32_t`, den nur dieser Worker schreibt. `last` und `seen_at` gehören dem Supervisor. `armed` wird eins, sobald die erste Runde angekommen ist. Vorher entscheidet das Startup-Budget, ob der Hund schon laufen darf und ob fehlende Meldungen schon Fehler sind. Wann der Watchdog gestartet wird, muss in dieses Budget passen: Initialisierung und erste Runden müssen vor dem ersten möglichen Ablauf fertig sein. Danach vergleicht die Subtraktion ohne Vorzeichen das Alter mit `fresh`. Ein Wrap der Supervisor-Zeit ist über das übliche Tick-Intervall definiert, solange das Alter kürzer als eine volle Zählerperiode ist. Der Worker-Zähler selbst darf umlaufen. Der Vergleich ist „ungleich“, nicht „größer“.

Zwei Lesevorgänge zweier Worker sind kein atomares Paar. Jeder wird für sich bewertet. Ein Event-Bitset mit Exchange wäre ein anderes Muster und nur zulässig, wenn der Zugriff die Atomarität wirklich hat. „Lesen und später alles löschen“ verschluckt neue Bits. Das Fristenmodell löscht den Worker-Zähler nicht. Kommunikationsverzug, ein Neustart mitten in der Frist und das Intervall, in dem der Supervisor selbst nicht läuft, bleiben Annahmen. Sie stehen neben der Zahl `fresh`, nicht unsichtbar darin. Der IWDG mit ausgeschaltetem Fenster sieht zusätzlich nicht, ob absurd oft gefüttert wird.

Diese Lücke ist der nächste Punkt.

Folie 31 / 53

Die Schwäche des IWDG

Hauptfall dieser Vorlesung: IWDG mit ausgeschaltetem Fenster. Eine Schleife, die alle paar Millisekunden füttert, hält den Zähler am Leben, auch wenn die eigentliche Arbeit nur noch aus dem Füttern besteht.
Eine konfigurierte IWDG-Fensterfunktion kann frühes Füttern zusätzlich ablehnen. Das ist ein anderer Betriebszustand, keine Eigenschaft jedes IWDG-Bausteins.
Der Vergleich mit dem WWDG braucht dessen Takt, Fenster und die gewünschte Reaktionszeit. Beides gleichzeitig „strenger“ zu nennen, ohne diese Zahlen, ist leer.

Notizen 31 / 53

Timeout bedeutet: zu spät ist sichtbar, zu früh ist es im fensterlosen Betrieb nicht. Der Hund wird immer wieder nachgeladen, bevor 501 ms Nennzeit ablaufen. Fachlich kann die Schleife leer sein. Darum hängt der Nutzen an der Policy und, wenn gewollt, an einem Fenster. Das Fenster des IWDG auf dem H755 nicht mit dem Satz „IWDG kann das nie“ abtun. Wir haben es im Hauptbeispiel ausgeschaltet und benennen den Vergleich so.
Der WWDG zwingt das Füttern in ein Zeitfenster. Dafür hängt er am APB-Takt, nicht am LSI, und seine Frist ist kurz. Welche Reaktion die Anwendung braucht, entscheidet, ob der lange unabhängige Timeout, das kurze Fenster oder beide gemeint sind. Beide lösen auf diesem Baustein einen System-Reset aus. Sie sind keine automatisch unabhängige Sicherheitsarchitektur. Ein falsch informierter Supervisor kann beide füttern.
Der APB-getaktete Fensterwächter ist der nächste Baustein.

Folie 32 / 53

Typ 2: Window Watchdog (WWDG) Einführung

WWDG1 an APB3 in D1. WWDG2 an APB1 in D2. Das M4-Lehrteil benutzt WWDG2.

$$t_{\text{tick}} = 4096 \times P / f_{\text{PCLK}}$$

Lehrannahme, kein Messwert: $f_{\text{PCLK}} = 100\,\text{MHz}$, $P = 8$. P ist der Teiler, nicht der Zahlenwert eines HAL-Makros.

$$4096 \times 8 / 10^8 = 0{,}32768\,\text{ms}$$

Bei 50 MHz: 0,65536 ms. Nicht 200 MHz. Die zulässige APB-Grenze hängt von Bauteil und VOS ab. VOS ist der Spannungs- und Leistungsmodus.

Notizen 32 / 53

PCLK, Peripheral Clock, ist der Takt des Busses dieser Instanz, nicht der CPU-Takt und nicht ein verdoppelter Timer-Takt. Für WWDG2 ist das APB1. Ob der Baum im Projekt 100 MHz oder 50 MHz liefert, ist offen. Die Rechnung bleibt als Lehrannahme gekennzeichnet. Der Teiler 8 entsteht aus der Kodierung im Register. Ein Makro wie ein Prescaler-Symbol kann einen Bitfeldwert tragen, der nicht gleich 8 ist. Wir rechnen mit dem wirksamen Teiler 8 und prüfen das Symbol am Header.
4096 mal 8 ist 32768. Geteilt durch 100 Millionen ist 0,00032768 s, also 0,32768 ms je Abwärtsschritt. Bei halbem PCLK verdoppelt sich der Schritt. Ein APB von 200 MHz übernehmen wir nicht. Das Datenblatt begrenzt die Busfrequenzen abhängig von VOS. Die konkrete Grenze steht in DS12919 zur gelesenen Revision und zur gewählten Spannung. Sie ist keine Folienkonstante über 120 MHz hinaus, und 100 MHz bleibt eine Annahme innerhalb eines noch zu prüfenden Baums.
Aus dem Tick werden Reload und Window erst das erlaubte Intervall.

Folie 33 / 53

Das Fenster-Konzept (Window Watchdog)

Lehrkonfiguration, überall dieselbe: WWDG2, Teiler 8, T startet bei 127 = $0\mathrm{x}7\mathrm{F}$, Fenster $W = 80$ = $0\mathrm{x}50$, EWI bei $T = 64$ = $0\mathrm{x}40$, Reset beim Übergang nach $T = 63$ = $0\mathrm{x}3\mathrm{F}$. Erlaubt vor dem Refresh: der Zähler ist größer als $0\mathrm{x}3\mathrm{F}$ und höchstens gleich W. Bei $T = 80$ ist die obere Grenze erreicht und der Refresh erlaubt, nicht „noch zu früh“.
Idealisiert, erster Schritt einen vollen Tick nach dem Nachladen. Der frei laufende Teiler kann den ersten Schritt früher auslösen. Das Band ist bis zu etwa ein Tick.

Schritt	T	t ideal
0	127	0 ms
47	80	15,401 ms
48	79	15,729 ms
54	73	17,695 ms
63	64	20,644 ms
64	63	20,972 ms

5 ms: zu früh. 18 ms: nominal $T = 73$, praktisch PCLK, Tick und Pfad prüfen. 25 ms ohne Refresh: der Reset liegt vorher, wenn der Hund läuft und nicht einfriert. `osDelay(n)` sind n Kernel-Ticks, nicht n ms.

Notizen 33 / 53

RM0399 Rev. 5, Abschnitte 47.3 und 47.3.4, erlaubt den Refresh, wenn der Zähler größer als $0\mathrm{x}3\mathrm{F}$ und höchstens gleich dem Window ist. Gleichheit mit 80 ist hardwareseitig erlaubt. Im normalen Betrieb zielen wir nicht auf diese Kante und nicht auf 64. $T = 64$ liegt noch vor dem Reset und kann bei eingeschaltetem EWI die Unterbrechung schon ausgelöst haben. Der Reset folgt einen Zählschritt später, bei fortlaufendem Takt, auch wenn die absolute Zeit wegen der Teilerphase verschoben ist.

Notizen, Fortsetzung 33 / 53

Die Tabelle setzt $t = n \times 0{,}32768$ ms an. 47 mal 0,32768 ist 15,40096 ms. Das ist der idealisierte Zeitpunkt, an dem T 80 erreicht, also die obere erlaubte Grenze, nicht ein verbotener Bereich. 16 Schritte von 80 nach 64 sind nur der untere Teil bis zum EWI, nicht die ganze Zeit bis zum Reset. Der Reset ist der 64. Schritt.
Nach 18 ms ist bei dieser Anfangsphase der 54. Schritt vorbei, $T = 73$, der 55. noch nicht. Eine andere Teilerphase kann einen Schritt früher liegen. „Etwa 72“ ohne Phase ist keine Rechnung. 5 ms liegen bei etwa 15 Schritten, T noch weit über 80, Reset ohne EWI. 25 ms liegen hinter Schritt 64.
`osDelay` wartet Kernel-Ticks. Die Frequenz kommt aus `osKernelGetTickFreq` oder der Konfiguration. Der Start relativ zur Tickflanke, das Aufwachen und Präemption verschieben den Refresh. Ein GPIO-Marker direkt am Refresh und eine benannte Zeitbasis gehören zur Beobachtung. Eine LED genügt für die Fenstergrenze nicht.
Die drei Verläufe zeichnet die nächste Folie im selben Schrittmaß.

Folie 34 / 53

Visualisierung des WWDG-Fensters

Abbildung G. Einheit: Zählschritte der Lehrkonfiguration. Millisekunden nur mit $0{,}32768$ ms und der Phasenannahme. Kein EWI vor dem frühen Reset.



Gleichheit $T = 80$ bei Schritt 47 ist erlaubt. Der frühe Reset im ersten Streifen liegt davor, ohne EWI.

Notizen 34 / 53

Die drei Spuren haben dieselbe Schrittachse. Der erste Refresh bei Schritt 15 liegt vor Schritt 47, also bei $T > 80$. Die Hardware setzt dann zurück und wartet nicht auf $T = 64$. Darum fehlt dort der EWI-Marker. Der zweite Streifen füttert bei Schritt 54, $T = 73$, innerhalb $63 < T \leq 80$. Der Zähler beginnt neu bei 127. Der dritte füttert nicht. Bei Schritt 63 steht $T = 64$, das EWI-Ereignis, wenn es eingeschaltet und der IRQ bedient ist. Bei Schritt 64 folgt der Reset.
Die Achse ist das idealisierte Modell „erster Abwärtsschritt einen vollen Tick nach dem Nachladen“. Der Teiler läuft schon. Die Phase beim Registerschreiben ist unbekannt. Jedes Ereignis kann bis zu etwa einen Tick früher liegen. Das ist ein Band um die Tabelle, keine zweite exakte Kurve. Software, `osDelay` und Präemption kommen zusätzlich dazu. Die Millisekunden der vorigen Folie gelten nur für 100 MHz und Teiler 8.
Beide Hunde nebeneinander zu legen, ändert ihre Taktquellen nicht.

Folie 35 / 53

IWDG und WWDG im Doppelpack

Getrennte Takte und verschiedene Zeitbedingungen. Keine automatisch unabhängige Sicherheitsarchitektur.
Beide können einen System-Reset auslösen. Die RCC-Flags und der Debug-Freeze sind je Instanz zu sichern, sonst erklärt der Reset den falschen Hund.
Ein Supervisor kann beide falsch informieren. Im Labor zuerst eine Instanz, die andere ausdrücklich aus oder unbenutzt. Die Kombination ist optional und erst danach.

Notizen 35 / 53

Der IWDG im Hauptbeispiel sieht zu spätes Füttern über den LSI, mit ausgeschaltetem Fenster. Der WWDG sieht zu frühes und zu spätes Füttern über den APB und kann vorher einen IRQ liefern. Zusammen decken sie zwei Zeitbedingungen ab. Sie bleiben abhängig von derselben Policy, wenn dieselbe Aufgabe beide füttert. Ein blinder Timer, der beide bedient, verdeckt denselben Hänger zweimal.
Weil beide System-Reset erzeugen, reicht nach dem Neustart ein einziges unspezifisches „es hat zurückgesetzt“ nicht. Wir lesen die Resetursachen, bevor sie gelöscht werden, und notieren, welcher Hund einfrieren durfte. Ein gesetztes Flag aus einem früheren, nicht gelöschten Lauf ist nicht automatisch dieser Versuch. Die Historie kann mehrere Bits tragen.
Der Laborweg bleibt einzeln: erst das Fenster verstehen, dann optional den zweiten Hund. Welche Fehler das Fenster nicht sieht, zeigt die nächste Gegenüberstellung.

Folie 36 / 53

Warum WWDG? (Fehlerhafte Ausführung)

Eine Schleife, die sofort wieder füttert, scheitert am Fenster und bestünde einen reinen Timeout. Eine Schleife, die nach einer festen Wartezeit im Fenster füttert, besteht den WWDG und kann fachlich falsch sein. Ein übersprungener Rechenschritt löst keinen frühen Refresh aus, wenn danach dieselbe Pause kommt.
Der Fensterwächter ist ein zeitlicher Plausibilitätsmechanismus. Er ist kein Beweis des Kontrollflusses und kein Beweis des Ergebnisses. Dafür bleiben die Fortschritts- und Plausibilitätsprüfungen zuständig.

Notizen 36 / 53

Der Sinn des Fensters ist die untere und die obere Zeitgrenze. Wer in einer engen Schleife nur noch füttert, kommt zu früh und fliegt beim WWDG heraus. Beim fensterlosen IWDG würde dieselbe Schleife den Zähler am Leben halten. Wer umgekehrt die Pause einhält und dazwischen den falschen Stellwert rechnet, liegt zeitlich im Fenster. Der Hund sieht die Pause. Er sieht den Stellwert nicht.
Ein ausgelassener Arbeitsschritt ist deshalb nicht automatisch ein früher Refresh. Die Policy muss den Schritt selbst zählen, wie im Supervisor, und nur dann füttern. Der WWDG prüft danach nur noch, ob

Notizen, Fortsetzung 36 / 53

dieser Entschluss in das Fenster fiel. Zu selten und zu oft werden sichtbar. Falsch, aber pünktlich, bleibt unsichtbar.
Kurz vor dem späten Ablauf kann ein IRQ noch eine kurze Reaktion starten. Das ist der EWI.

Folie 37 / 53

Das Sicherheitsnetz: EWI

EWI, Early Wakeup Interrupt, ist ein gewöhnlicher maskierbarer IRQ, kein NMI. NMI, Non-Maskable Interrupt, wäre nicht auf diesem Weg abschaltbar.
Bei der Lehrkonfiguration liegen zwischen $T = 64$ und $T = 63$ nominal $0{,}32768$ ms, also 327,68 µs. Davon gehen Eintritt, bereits verstrichene Phasenzeit und die Befehle ab. Das Softwarebudget ist kleiner. Eine gemessene Worst-Case-Latenz haben wir nicht.
Zu früher Refresh setzt zurück, ohne EWI. Ein maskierter IRQ, eine ungeeignete Priorität oder ein blockierter Handler kann den Callback verhindern. Im Resetversuch bei $T = 0x40$ nicht erneut füttern.

Notizen 37 / 53

Abbildung H trennt das Hardwareereignis, die Eintrittslatenz, die kurze Arbeit und den nächsten Zählschritt. Die 327,68 µs sind der ideale Abstand zweier Schritte bei 100 MHz und Teiler 8, nicht die Zeit, die der Callback sicher hat. Ein Teil kann schon verstrichen sein, wenn der Handler den ersten Befehl ausführt. Kein `printf`, kein Flash, kein Delay, kein langes Warten. RTOS-Aufrufe nur, wenn API und Priorität sie erlauben. Im vereinfachten hochpriorisierten Lehrpfad nutzen wir keine RTOS-API.
Ein hochpriorisierter Task sperrt einen freigegebenen, dringenderen Hardware-Interrupt nicht automatisch. Maskierung wäre eine höhere oder gleiche Priorität im NVIC-Sinn, eine globale IRQ-Sperre oder ein ausgeschalteter Interrupt. Die numerische Ordnung ist umgekehrt: eine kleinere Prioritätszahl ist dringender.
Refresh bei `0x40` ist hardwareseitig noch möglich, weil $T > 0x3F$ und, bei $W = 80$, $T \leq W$. Im Reset-Experiment unterlassen wir ihn bewusst. Ein anderes Recoverybeispiel darf dort füttern, muss aber eine andere Überschrift tragen und darf nicht gleichzeitig sagen, der Reset sei unvermeidlich.
Welche kurze Reaktion in das Budget passt, ist die Policy der nächsten Folie.

Folie 38 / 53

Nutzen des EWI (Safe Shutdown)

Lehrpolicy für das Reset-Experiment: Ursache merken, kurzen GPIO-Marker setzen, nicht erneut füttern, Reset zulassen.
Ein sicherer Anlagenzustand braucht eine definierte Anwendung, einen Sollpegel und eine maximale Reaktionszeit. Die haben wir hier nicht. Eine LED oder ein GPIO zeigt, dass der Callback lief. Sie ist kein Sicherheitsnachweis.
Pins können beim Reset hochohmig oder analog werden. Externe Pull-Widerstände und die Boot-Freigabe müssen den gewünschten Ruhepegel tragen. „Ausgänge auf Reset-Pegel“ ohne Schaltung ist keine Anweisung.

Notizen 38 / 53

Der Callback ist kein zweites Betriebssystem. Er darf nur tun, was in den restlichen Bruchteil des Tick-Abstands passt. Ein Marker an einem vorher konfigurierten Pin ist so eine Aktion. Logging in Flash, eine formatierte Ausgabe oder ein Warten auf den anderen Kern sind es nicht. Ein optionaler RAM-Satz überlebt nur, wenn die Speicherbank den Reset behält, der M7-Cache ihn nicht nur als Dirty-Line hält und der Start ihn nicht löscht oder als gültig missversteht. Ohne diesen Vertrag lassen wir den Satz weg.
Die Policy ist eindeutig: im Resetversuch kein zweiter Refresh. Wer bei $0\mathrm{x}40$ bewusst füttert, übt Recovery und muss den Zähler neu einordnen. Beides auf einer Folie als gleichzeitig unvermeidlich und vermeidbar zu bezeichnen, widerspricht sich.
Die Reaktionszeit der Anwendung wäre Frist des Nachweises plus Rest des Watchdogs plus Boot bis zur sicheren Freigabe. Der EWI verkürzt nur das Stück unmittelbar vor dem Reset, und nur wenn er wirklich läuft. Die LED beweist den Aufruf unter Laborbedingungen, nicht diese Kette.
Der HAL-Pfad muss zur Instanz WWDG2 passen.

Folie 39 / 53

HAL-API für den WWDG

Lehrfolge, ungebaut. Der Init des Referenztreibers kann den Zähler bereits starten. Die erste erlaubte Refreshzeit liegt dann im Fenster, nicht bei $T = 127$.

1. Takt der Instanz an, WWDG2 für den M4-Teil
2. Prescaler-Teiler 8, Window 80, Reload 127, EWI an
3. NVIC und den Handler dieser Instanz, nicht einen erfundenen Namen
4. Callback ohne RTOS, im Resetversuch ohne Refresh
5. GPIO-Marker direkt am Vorgang, den wir meinen

Rückgabewert prüfen. Debug-Freeze notieren.

Notizen 39 / 53

`HAL_WWDG_Init` schreibt die Konfiguration und kann den Abwärtszähler starten. Ab diesem Moment gilt das Fenster. Wer sofort füttert, ist zu früh und setzt zurück, ohne EWI. Der erste legale Refresh liegt bei der Lehrkonfiguration frühestens an $T = 80$, im Betrieb mit Abstand, zum Beispiel im inneren Bereich um Schritt 54. Die Zeit bis dorthin hängt von PCLK und Teilerphase ab. Wir beschriften sie mit der Tabelle, nicht mit einer alten festen Delay-Konstante aus einer anderen Anleitung.
Der IRQ-Name und die Handlerfunktion kommen aus dem Header der Instanz. Eine geratene `WWDG_IRQHandler`-Schreibweise ohne Zuordnung zu WWDG1 oder WWDG2 kann den falschen Vektor treffen. Der Callback erhält die frühe Warnung. Er quittiert den EWI-Status so, wie der Treiber es verlangt, und füttert im Resetversuch nicht. Die NVIC-Priorität liegt im Lehrpfad so, dass keine RTOS-API nötig ist.
Ein Markerpin wird im selben Kontext umgeschaltet, in dem der Refresh oder der EWI-Eintritt stattfindet. Eine LED, die der Scheduler später zeigt, datiert das Ereignis nicht. `osDelay` bleibt in der Task und nur mit bekannter Tickfrequenz. Ohne Messung bleiben die Zeiten analytische Erwartungen.
Die Eigenschaften beider Hunde stehen danach in einem Beispiel nebeneinander.

Folie 40 / 53

Zusammenfassung: Watchdogs

- 501 ms sind der IWDG-Nennwert bei 32 kHz, Reload 500, Teiler 32, Fenster aus
- WWDG2, Lehr-PCLK 100 MHz, Teiler 8, Reload 127, Window 80
- Füttern ist erlaubt, nachdem der Entwurf einen frischen Nachweis hat
- Die Hardware prüft diese Bedeutung nicht
- EWI ist ein maskierbarer Schritt vor dem späten Reset, nicht vor dem frühen

Frage: Warum übersieht Idle einen Deadlock der Worker? Weil Idle weiterläuft. Frage: Warum kann der Reset ohne EWI kommen? Weil zu frühes Füttern nicht auf $T = 64$ wartet.

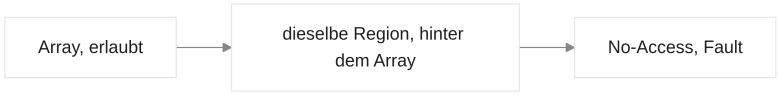
Notizen 40 / 53

Ein zusammenhängendes Bild: Der Supervisor füttert den WWDG2 nur, solange jeder nötige Worker frisch ist und der Zähler im Fenster liegt. Der IWDG mit ausgeschaltetem Fenster bleibt der lange Rückenfang, wenn dieser Pfad ganz ausbleibt, und nur unter der Nennrechnung von 501 ms plus Datenblattgrenzen. Der EWI gibt im späten Fall einen kurzen Marker und lässt den Reset zu. Die Hardware versteht „frischer Nachweis“ nicht. Sie versteht nur, ob der Refresh in ihrer Zeitbedingung lag.
Die zwei Fragen haben kurze Antworten. Idle beweist den eigenen Pfad, nicht die blockierten Tasks und nicht einen Tick. Der frühe Fensterreset passiert, bevor der Zähler 64 erreichen muss. Speicherfehler brauchen einen anderen Mechanismus als diese Frist.
Die MPU prüft Regionen, nicht die Absicht des Programms.

Folie 41 / 53

Speicherschutz: Die MPU (Erinnerung)

Die MPU prüft konfigurierte Regionen des Kerns, der sie trägt. Sie kennt keine C-Arraylänge. Abbildung I. Das Array liegt in einer größeren erlaubten Region. Ein Zugriff hinter dem Array, aber in der Region, bleibt erlaubt. Erst die No-Access-Region verletzt das Recht.



Cache-Attribute aus VL6 sind kein Zugriffsschutz. DMA und der andere Kern brauchen eigene Maßnahmen.

Notizen 41 / 53

Die Watchdogs haben die Zeit geprüft. Ein Zeiger kann trotzdem neben das Array zeigen. Die MPU, Memory Protection Unit, vergleicht die Adresse eines CPU-Zugriffs mit Regionen: Basis, Größe, Rechte und Attribute. Liegt der Zugriff in einer erlaubten Region, ist er aus Sicht der MPU legal, auch wenn er in C hinter dem letzten Element liegt. Die höhere Regionsnummer gewinnt bei Überlappung. Das ist die Armv7-M-Regel und gilt für beide Kerne jeweils auf ihrer eigenen MPU.
Abbildung I zeichnet drei Orte in einer Adressrichtung: das Array, den Rest derselben erlaubten Region und eine anschließende verbotene Region. Nur der dritte Pfeil ist der MPU-Verstoß. Die Cache-Bits C, B und S aus Vorlesung 6 sagen, ob zwischengespeichert und gebuffert wird. Sie sind nicht die No-Access-Rechte. DMA ist ein eigener Busmaster und läuft an dieser CPU-MPU vorbei. Der andere Kern hat eine eigene MPU oder keine. Eine Region auf dem M4 schützt den M7 nicht.
Ein erkannter Verstoß wird zur Exception, nicht zu einer Task-Fehlermeldung.

Folie 42 / 53

Der primäre Zweck der MPU

Kette: verbotener CPU-Zugriff, MemManage, wenn die Exception freigegeben ist und nicht eskaliert, sonst HardFault.
Das ist kein Halt in einer festen Nanosekunde und kein sicherer Stillstand. Exception-Stacking kann scheitern, wenn der Stack schon zerstört ist. Ausgangsreaktion und Wiederanlauf sind zusätzliche Maßnahmen.

Notizen 42 / 53

MemManage ist die Exception für einen MPU-Verstoß, sofern sie eingeschaltet ist. HardFault ist die härtere Exception, in die ein Fault eskalieren kann, zum Beispiel wenn die MemManage-Exception gesperrt ist oder während der Annahme selbst ein Fehler auftritt. „Fault-Maske“ als ein Wort erklärt das nicht. Gemeint sind konkrete Bits: Exception aktiv, Priorität, und ob FAULTMASK oder ein NMI die Annahme verändert. Die genaue Eskalation steht in PM0214 für den M4 und PM0253 für den M7.
Der Kern nimmt die Exception an, nachdem der Zugriff geprüft wurde. Das ist schnell, aber keine zugesagte

Notizen, Fortsetzung 42 / 53

Nanosekunden-Obergrenze aus diesem Kurs. Danach läuft Handler-Code, nicht „alles steht sicher“. Wenn der Stackpointer schon außerhalb des benutzbaren Stacks liegt, kann das Sichern der Register beim Exception-Eintritt erneut fehlschlagen. Dann hilft die Guard nicht mehr beim geordneten Beenden einer Task. Nur die fehlerhafte Task zu beenden, wäre eine zusätzliche Recovery des RTOS. Der gewöhnliche Port tut das nicht von selbst.
Ein reproduzierbarer Test braucht eine erlaubte Annahme der Exception und einen Stack, der für diesen Fault noch tragfähig ist. Adresse 0 ist dafür ein schlechter Beweis.

Folie 43 / 53

Use-Case 1: Null-Pointer-Protection

NULL in C ist ein Nullzeiger. Ihn zu dereferenzieren ist undefiniertes Verhalten, kein portabler Test.
Ein uninitialisierter Zeiger ist nicht automatisch NULL. Adresse 0 kann auf dem H755 eine Abbildung von ITCM oder Boot sein. ITCM ist Instruction Tightly Coupled Memory. Wir raten diese Abbildung nicht.
Reproduzierbarer Test: eine dokumentierte Adresse in einer No-Access-Region, Disassembly und Faultstatus prüfen. Eine erlaubende Region mit höherer Nummer schluckt den Zugriff.

Notizen 43 / 53

Der verbreitete Satz „NULL fängt die MPU ab“ mischt drei Dinge. Das C-Makro NULL bezeichnet einen Nullzeiger. Was der Compiler daraus für einen Maschinenzugriff macht, ist bei einer Dereferenz undefiniert. Er muss nicht die Adresse 0 lesen. Ein Zeiger, der nie gesetzt wurde, enthält nicht zuverlässig 0. Und die Adresse 0 im Adressraum des STM32H755 ist nicht „garantiert leer“. Sie kann eine Code-Abbildung sein. Wer das als Test benutzt, prüft eine Vermutung.
Der Laborzugriff geht auf eine Adresse, die in der Regionstabelle ausdrücklich No-Access ist, ausgerichtet ist und von keiner höheren Region erlaubt wird. Danach sehen wir im Disassembly den Load oder Store und im Faultstatus die Adresse. Trifft der Zugriff nicht ein, war die Region nicht aktiv, die Priorität einer Überlappung anders, oder der Compiler hat den Zugriff entfernt. `volatile` auf einem Testobjekt kann das Wegoptimieren erschweren. Es ist kein Ersatz für das Disassembly.
Frage: Warum sieht die MPU nicht jeden Arrayüberlauf? Weil der Zugriff in der erlaubten Region bleiben kann. Die Länge des Arrays steht nicht in der MPU.
Die Stackgrenze ist ein zweiter, ähnlich begrenzter Anwendungsfall.

Folie 44 / 53

Use-Case 2: Stack-Overflow Erkennung

Der Stack wächst zu kleineren Adressen. Eine Guard liegt unterhalb, ausgerichtet, mit No-Access. Ein Sprung über sie oder ein Überlauf in eine erlaubte Nachbarregion bleibt unsichtbar. Abbildung J, Zugriffsmatrix bei aktiver passender No-Access-Region:

Zugriff	durch diese Guard?
unprivilegierter Thread	ja
privilegierter Thread	ja
gewöhnlicher Handler	ja
HardFault, NMI, FAULTMASK	nur mit HFNMIENA
DMA	nein
anderer Kern	nein

Der gewöhnliche FreeRTOS-CM4-Port legt nicht je Task eine Guard an. Eine feste Guard kann man trotzdem von Hand setzen. Dynamisch je Task braucht den MPU-Port oder eine eigene Umschaltung.

Notizen 44 / 53

SP, Stack Pointer, zeigt auf die aktuelle Stackstelle. MSP und PSP sind Main und Process Stack Pointer. Welcher im Thread Mode gilt, hängt vom Port ab. Der Handler Mode benutzt den Main Stack. Die Zeichnung meint: die benutzten Adressen werden kleiner, die Guard liegt an der unteren Kante, und ihre Größe und Ausrichtung müssen zur MPU passen. Ein Zugriff, der die Guard überspringt und in einer darunter erlaubten Region landet, ist kein Treffer.
Die frühere Aussage „nur die Task-Spalte sieht die Guard“ ist falsch. No-Access gilt für privilegierte und unprivilegierte Zugriffe dieses Kerns, einschließlich gewöhnlicher Interrupt-Handler, solange die Region aktiv und die MPU eingeschaltet ist. Thread Mode und Handler Mode sind nicht dasselbe wie privilegiert und unprivilegiert. Gewöhnliche Handler sind privilegiert und trotzdem nicht von der Region ausgenommen. HFNMIENA entscheidet, ob die MPU auch während HardFault und NMI sowie bei gesetztem FAULTMASK wirkt. PRIVDEFENA ist der privilegierte Hintergrund für Adressen, die keine aktive Region trifft. Er hebt eine explizite No-Access-Region nicht auf. DMA und der andere Kern stehen außerhalb dieser MPU.
Eine manuelle Guard an einer festen Adresse ist möglich, ohne dass jede Task ihre eigene Region beim Kontextwechsel erhält. Beides nicht vermischen. Ein Fault beendet die Task nicht automatisch.

Folie 45 / 53

Privileged vs. User Mode

Modus	Privileg	MPU-Region
Thread, unprivilegiert	nein	aktive Region gilt
Thread, privilegiert	ja	aktive Region gilt
Handler	ja, gewöhnlich	aktive Region gilt

PRIVDEFENA füllt nur die Lücken ohne Region. Ein Task-Fault lässt den Rest nicht automatisch gesund weiterlaufen. Das wäre Recovery und ist hier nicht gebaut.

Notizen 45 / 53

Die Matrix der vorigen Folie braucht die zwei Achsen, die oft zusammengeworfen werden. Der Thread Mode kann privilegiert oder unprivilegiert sein. Der Handler Mode der gewöhnlichen Interrupts ist privilegiert. Privilegiert heißt: Zugriff auf geschützte Systemregister und, falls PRIVDEFENA gesetzt ist, auf den Hintergrundadressraum dort, wo keine Region liegt. Es heißt nicht: die No-Access-Region ignorieren.
Der FreeRTOS-Port kann Tasks unprivilegiert laufen lassen und Systemdienste über einen Wrapper in den privilegierten Code führen. Ob der vorhandene CM4-Port das tut, steht in seinen Quellen, nicht in einem beliebigen MPU-Port. Wir übertragen das nicht. Solange der Port keine geprüfte Recovery hat, endet ein Fault nicht mit dem Satz „nur diese Task ist tot“. Der Stack des Faults, die Priorität und ein möglicher zweiter Fault entscheiden, ob der Kern noch eine Schleife in HardFault erreicht oder hängen bleibt.
Damit sind die drei Schutzfragen getrennt: wer darf gleichzeitig schreiben, wer rechtzeitig fertig wird, und welche Adresse dieser Kern anfassen darf.

Folie 46 / 53

Zusammenfassung MPU & Safety

- Die MPU prüft Regionen eines Kerns, nicht C-Längen
- Ein Fault ist eine Exception, kein sicherer Zustand
- DMA und der andere Kern brauchen eigene Maßnahmen
- HSEM, Watchdog und MPU beantworten drei verschiedene Fragen

Frage: Warum ist die Notification kein Besitz? Weil Take danach neu entscheiden muss. Frage: Warum übersieht Idle blockierte Tasks? Weil Idle selbst läuft. Frage: Warum erzeugt ein Überlauf in einer erlaubten Region keinen Fault? Weil die Region den Zugriff erlaubt.

Notizen 46 / 53

Ein Fehlerfall verbindet die Grenzen, ohne sie zu vermischen. Zwei Kerne inkrementieren nur unter HSEM und mit der Sichtbarkeit aus Vorlesung 6. Der Supervisor füttert nur bei frischen Nachweisen und im

Notizen, Fortsetzung 46 / 53

Fenster. Die MPU fängt einen CPU-Zugriff in eine No-Access-Region dieses Kerns ab, einschließlich eines gewöhnlichen privilegierten Handlers. Sie fängt den anderen Kern, DMA und einen Überlauf innerhalb einer erlaubten Region nicht ab. Der Fault ersetzt nicht den definierten Ausgangszustand einer Anlage. Die drei Fragen wiederholen die häufigsten Verwechslungen und haben dieselben Antworten wie an ihrer Herkunftsstelle. Exklusiver Zugriff, zeitlicher Fortschritt und zulässige Adressen bleiben drei Nachweise. Das Labor beobachtet sie getrennt und trägt keine erfundenen Messwerte ein.
Die Vorbereitung nennt die offenen Projektfelder neben den festen Lehrwerten.

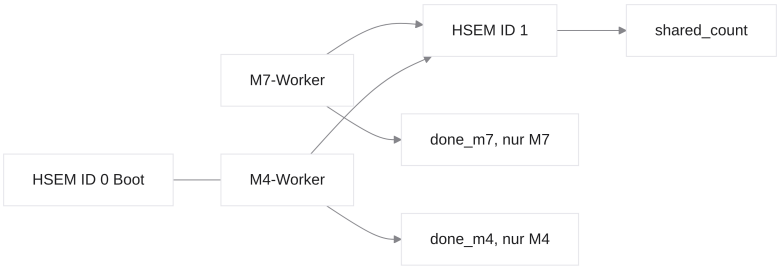
Folie 47 / 53

Laborvorbereitung: Multicore-Safety

Drei Versuche: Zähler, ein Watchdog, EWI und MPU. Nicht gleichzeitig alles einschalten.

Punkt	Lehrannahme	im Projekt prüfen
Board	nur NUCLEO-H755ZI-Q, wenn es das ist	Aufdruck
HSEM Boot	ID 0, wenn das Template sie nutzt	Startup
HSEM Zähler	ID 1, wenn frei	Belegung
Shared	`ipc`-Bereich, nicht cachebar	Map, MPU
ProcessID	M7 `0x37`, M4 `0x34`	ein Worker je Kern
WWDG	WWDG2, PCLK-Annahme 100 MHz	Taktbaum
Resetflags	früh lesen	vor HAL-Löschen

Abbildung K. Besitz und Veröffentlichung sind verschiedene Pfeile.



Zähler einmal auf 0, bevor beide starten. Endliches N mit $2N$ in `uint32_t`. Ein ungeschützter Lauf, der $2N$ trifft, beweist keine Racefreiheit.

Notizen 47 / 53

Abbildung K zeigt zwei Worker, zwei HSEM-IDs und getrennte Fertigfelder. Der Boot bleibt ID 0, wenn Vorlesung 6 ihn so benutzt. Der Zähler benutzt ID 1 nur nach einer Prüfung, dass niemand sonst sie hält. `shared_count` liegt im nicht cachebaren Shared-Bereich. Jeder Kern schreibt sein eigenes Done- und Fehlerfeld. Der andere liest es mit dem Sichtbarkeitsvertrag, er schreibt es nicht mit. Lokale Zeitbasen bleiben getrennt: HAL-Tick auf dem M7, Kernel-Tick auf dem M4. Messpins sind nicht die LED. N ist eine kleine Lehrgrenze. $2N$ darf den 32-Bit-Bereich nicht verlassen. Der Zähler wird genau einmal initialisiert, bevor beide Worker freigegeben werden. Danach schreiben sie ihn nur innerhalb von HSEM.

Notizen, Fortsetzung 47 / 53

Die Erwartung nach zwei erfolgreichen Fertigmeldungen ist $2N$. Vorher ist jeder gelesene Wert ein Zwischenstand. Ein Lauf ohne Schloss, der zufällig $2N$ ergibt, ist kein Beweis. Ein getrennter didaktischer Lauf darf die Kollision zeigen und muss anders beschriftet sein.
Der vollständige Ablauf beider Worker folgt als eigener Schritt.

Folie 48 / 53

Schritt 2: Der asynchrone Multicore-Zugriff

Lehrcode, ungebaut. Nur erfolgreiche Inkremente zählen. Ein Fehlschlag ist kein Schritt.

```
uint32_t hits = 0, t0 = HAL_GetTick();
while (hits < N) {
    if ((HAL_GetTick() - t0) > 2000u) {
        m7_fail = 1;
        break;
    }
    if (HAL_HSEM_Take(1, 0x37u) != HAL_OK)
        continue;
    __DMB();
    shared_count += 1u;
    __DMB();
    HAL_HSEM_Release(1, 0x37u);
    hits++;
}
__DMB();
m7_done = hits;
```

Der M4 tut dasselbe mit `0x34`, `osKernelGetTickCount` und `osDelay(1)` außerhalb des Schlosses. 2000 Kernel-Ticks sind nicht generell 2 s. Die HAL-Zeitbasis des M7 ist eine andere Uhr. Das Gesamtbudget muss zu N und den Wartezeiten passen.
Gelesen wird `shared_count` erst, wenn beide Done-Felder den Erfolg melden und die Deadline hält. Sonst ist der Versuch ungültig.

Notizen 48 / 53

Die Timeoutzeile ist vollständig: bei Überschreitung Fehlerfeld setzen und die Schleife verlassen, nicht den Sollwert $2N$ erfinden. `HAL_GetTick` zählt die Zeitbasis der HAL. Oft ist sie 1 ms. Das ist eine Projektprüfung, keine zweite Systemuhr des M4. `osKernelGetTickCount` zählt Kernel-Ticks. Deren Länge liefert `osKernelGetTickFreq`. `osDelay(1)` gibt für einen Tick ab und steht nicht zwischen Take und Release.
Der M4-Backoff mit `osDelay` ist erlaubt und nicht fair. HSEM hat keine Warteschlange. Ein globaler Versuchsrahmen muss so groß sein, dass N erfolgreiche Schritte und die legalen Wartezeiten hinein-

Notizen, Fortsetzung 48 / 53

passen. Sonst erzeugt das Budget selbst den Abbruch. Jeder Kern veröffentlicht sein Done-Feld als alleiniger Schreiber, mit Barriere, in der nicht cachebaren Region. Der Auswerter wartet bis zu seiner Deadline auf beide Erfolge. Dann liest er den Zähler und vergleicht mit $2N$. Ein Abbruch, ein fehlendes Done oder ein Überlauf machen den Vergleich ungültig.
Der Lost-Wakeup-Versuch ist ein zweiter, getrennter Ablauf mit Notification, Flag, Rearming und einer Gesamtdeadline. Er ist nicht diese Zählschleife. Mehrere Tasks an ID 1 bräuchten zusätzlich den lokalen Mutex. Den einfachen Versuch lassen wir bei einem Worker je Kern.
Frage: Wann darf man mit $2N$ vergleichen? Nach beiden erfolgreichen Fertigmeldungen, gültiger Sichtbarkeit, ohne Abbruch und ohne Überlauf.
Danach kommt ein Watchdog, zunächst nur einer.

Folie 49 / 53

Schritt 3: Den WWDG integrieren

Tabelle zuerst, Delays danach. Lehrannahme, bis der Taktbaum steht:

Größe	Wert
Instanz	WWDG2
PCLK	100 MHz, Annahme
Teiler	8, nicht das HAL-Makro als Zahl
Reload, Window	127, 80
EWI, Reset	64, Übergang nach 63
Tick	0,32768 ms, Idealphase

Clock, Init, NVIC, Handler der Instanz, Callback. Debug-Freeze und frühe Resetflags dokumentieren. Andere Watchdogs in diesem Versuch aus oder als unbenutzt eintragen. Alte feste Delays aus einer anderen Anleitung nicht übernehmen.

Notizen 49 / 53

Die Initialisierung startet den Zähler möglicherweise schon in `HAL_WWDG_Init`. Die erste erlaubte Fütterung liegt im Fenster, nicht im selben Moment. Wir füllen die Tabelle aus dem tatsächlichen PCLK, sobald er bekannt ist, und rechnen die Schritte neu. Solange 100 MHz eine Annahme bleibt, bleiben 15,4 ms und 17,7 ms analytische Erwartungen dieser Annahme. Ein Delay von 5 ms, 15 ms oder 50 ms aus einer älteren Anleitung passt dazu nicht automatisch, und `osDelay` zählt außerdem Ticks.
Die IRQ-Priorität und der Handler gehören zu WWDG2. Ein Freeze im Debugger unterdrückt den Reset und macht den Lauf zu einem anderen Experiment. Die Resetursache lesen wir, bevor eine Init-Funktion sie löscht. IWDG1, IWDG2 und WWDG1 schreiben wir in das Protokoll als aus oder nicht benutzt, damit ein gesetztes Flag nicht still einer anderen Instanz gehört.
Die Fälle danach laufen einzeln, jeder mit eigener Erwartung und leerer Beobachtung.

Folie 50 / 53

Schritt 4: Den Wachhund füttern & Crashen

Abbildung L. Analytische Erwartung. Beobachtung bleibt leer, bis ein Lauf sie füllt. Kein erfundener Messwert.

Fall	Erwartung	Resetursache	Zeit	Deutung
Refresh zu früh	Reset, kein EWI nötig	offen	offen	offen
Refresh im Fenster	kein Reset durch diesen Kick	offen	offen	offen
kein Refresh	EWI nur wenn IRQ läuft, dann Reset	offen	offen	offen
CPU-Hog, niemand füttert	Reset	offen	offen	offen
Idle füttert, Worker blockiert	Hund kann schweigen	offen	offen	Policy zu schwach
Frist abgelaufen	kein weiterer normaler Kick	offen	offen	offen

Marker am GPIO direkt am Refresh. Die LED ist kein Mikrosekundenmaß.

Notizen 50 / 53

Jeder Fall beginnt aus einer bekannten Konfiguration. Der frühe Refresh erwartet den Reset wegen der Fensterverletzung und setzt keinen EWI voraus. Der gültige Refresh erwartet, dass dieser Kick selbst nicht zurücksetzt. Ob er wirklich im Fenster lag, sagt erst der Marker gegen die dokumentierte Zeitbasis. Ohne Refresh kann das EWI kommen, wenn es eingeschaltet ist und der IRQ bedient wird. Danach folgt der Ablauf-Reset, sofern der Hund nicht einfriert. Ein CPU-Hog setzt nur zurück, wenn kein anderer Pfad weiter blind füttert. Das Idle-Beispiel ist absichtlich die schwache Policy: blockierte Worker und weiterlaufendes Füttern. Es darf nicht als „der Watchdog hat den Deadlock erkannt“ protokolliert werden. Der Supervisor-Fall beendet den Refresh, wenn die Frischefrist abgelaufen ist. Die Reaktionszeit ist Frist plus Rest des Zählers plus Boot, nicht der EWI allein.
Die Resetursache wird früh gesichert. Mehrere gesetzte Bits sind eine Historie, kein einzelnes automatisches Urteil. Debugger-Freeze, tatsächlicher PCLK und Tickfrequenz gehören in die Bedingungen. Eine LED, die erst der Scheduler zeigt, datiert das Fenster nicht.
Die kurze Reaktion und der MPU-Test sind getrennte letzte Schritte.

Folie 51 / 53

Schritt 5: Den Safe-Shutdown implementieren

EWI-Policy des Resetversuchs: kurzer Marker, nicht erneut füttern, Reset zulassen. Kein Flash, kein `printf`, kein Delay. Keine RTOS-API in diesem priorisierten Pfad.
Pinverhalten beim Reset nicht mit einem aktiven Ausgang vor dem Reset verwechseln. Die LED zeigt den Callback, nicht einen sicheren Anlagenzustand.
MPU-Versuch getrennt: No-Access-Region, dokumentierte Adresse, Regionsnummer, Privileg, Disassembly, Faultstatus. Kein `*NULL` als Beweis. Keine erfundene Task-Recovery. Die maximale Reaktionszeit ist eine Anforderung, kein automatisches Ergebnis des EWI.

Notizen 51 / 53

Der Callback bleibt in dem Budget, das vom nominalen Tick-Abstand übrig ist. Er setzt einen vorher konfigurierten Pin und kehrt zurück. Der Zähler läuft auf 63 und löst den System-Reset aus. Ein RAM-Log nur mit geprüftem Überleben und einem Start, der ihn nicht als gültigen Zustand missversteht. Eine industrielle Abschaltung ohne beschriebene Anlage und ohne Sollpegel ist nicht Teil der Beobachtung.
Der MPU-Versuch benutzt eine andere Vorbereitung. Die Region ist aktiv, No-Access, und keine höhere Region erlaubt die Testadresse. Der Zugriff ist ein sichtbarer Load oder Store im Disassembly. Wir wissen vorher, welcher Stack die Exception aufnehmen muss. MemManage oder HardFault wird notiert, einschließlich Eskalation. Danach behaupten wir nicht, dass die Task beendet wurde und der Rest gesund weiterläuft. Adresse 0 und ein Nullzeiger bleiben aus diesem Beweis draußen.
Die Reaktionszeit einer Anwendung wäre als Soll zu nennen und mit Frist, Watchdogrest und Boot zu vergleichen. Ein gelaufener EWI beweist diese Kette nicht. Die Schlussfolie beantwortet die vier Lernziele ohne einen Hardwarelauf zu erfinden.

Folie 52 / 53

Die Essenz des Tages

- Ein lokaler Mutex und `__disable_irq` schließen den anderen Kern nicht aus
- HSEM entscheidet den Besitz. Sichtbarkeit bleibt VL6. Notification ist kein Besitz
- 501 ms sind der IWDG-Nennwert. Das WWDG-Fenster gilt für Reload 127 und Window 80
- $T = 80$ ist erlaubt. EWI bei 64, Reset beim Schritt nach 63
- Die MPU prüft Regionen eines Kerns, auch für gewöhnliche privilegierte Handler
- Das Labor demonstriert. Im Repo gibt es keinen Hardwarelauf

Notizen 52 / 53

Die vier Ziele haben Antworten. Ein Lost Update entsteht, weil zwei Registerkopien von 5 jeweils 6 schreiben. Das Warten verliert die Freigabe nicht, wenn das Flag vor dem Wait stehen bleibt und Take danach den Besitz neu holt. Das Fenster rechnet aus dem APB-Tick der Lehrannahme, mit benannter Teilerphase. Die MPU-Grenze ist die Region, nicht die Arraylänge und nicht der andere Kern.

Notizen, Fortsetzung 52 / 53

Dokumentiert ist die System-Resetwirkung der vier Watchdogs. Offen bleiben die installierte HAL, der echte PCLK, die LSI-Grenzen zur gewählten Temperatur und die freie HSEM-ID. ProcessID $0\mathrm{x}37$ bleibt eine Lehrkennung. ReleaseAll bleibt kein Datenreparaturweg.
Frage: Warum reicht HSEM nicht gegen eine alte Cachekopie? Weil Besitz die Zeile nicht zurückschreibt. Frage: Warum ist 501 ms nicht garantiert? Weil der LSI streut. Frage: Ist $T = 80$ bei $W = 80$ zu früh? Nach der geprüften Beschreibung nein. Im Betrieb halten wir trotzdem Abstand.
Der nächste Termin ist die Capstone-Architektursicht: Muster über mehrere Kerne, nicht ein neues Schloss.

Folie 53 / 53

Glossar und offene Punkte, zu Folie 51

Nachschlagehilfe. Die lokale Erklärung auf der jeweiligen Folie bleibt maßgeblich.

Kurz	Rolle hier
HSEM	Hardware-Schloss, ID 0 Boot, ID 1 Zähler
ProcessID	8-Bit-Kennung, $0\mathrm{x}37$ und $0\mathrm{x}34$
DMB	ordnet Speicherzugriffe, reinigt keinen Cache
IWDG / WWDG	LSI-Timeout und APB-Fenster, beide System-Reset
EWI	maskierbarer IRQ bei $T = 64$
MPU	Regionen eines Kerns, kein DMA, kein fremder Kern
LSI / PCLK	32 kHz Nennwert, 100 MHz Lehrannahme

Offen: Boardrevision, HAL-Stand, Taktbaum, Tickfrequenz, freie HSEM-ID, LSI-Min/Max zur Datenblattzeile, Debug-Freeze. Quellen der Erklärung: RM0399 Rev. 5 Kapitel HSEM, WWDG und IWDG sowie RCC-Reset, DS12919 Rev. 3, der öffentliche STM32H7-HAL-Referenztreiber, PM0214 und PM0253. Keiner dieser Stände ist als Projektkopie im Repo nachgewiesen.

Notizen 53 / 53

Dieser Anhang gehört zur Schlussfolie. Er wiederholt keine Herleitungen. Wer eine Abkürzung auf einer früheren Folie trifft, findet sie dort im Satz und hier als Index. Die offenen Felder ändern die dokumentierte Resetwirkung nicht. Sie ändern die Millisekunden, sobald der PCLK nicht 100 MHz ist, und die IWDG-Grenzen, sobald die Datenblattzeile gewählt ist. Ein Build des Ziels und ein Lauf auf dem Board haben nicht stattgefunden.